

XCPC 算法模板

usedchang



2026 年 5 月 8 日

目录

1 杂项	2
1.1 IO 加速	2
1.2 <code>__int128</code> 操作	2
1.3 对拍	2
1.4 热身赛测试语句	3
1.5 编译指令	4
1.6 <code>cpp</code> 扩栈 (win)	4
2 数据结构	5
2.1 ST 表	5
2.2 并查集	5
2.3 线段树 (普通区间和查询)	6
2.4 笛卡尔树	6
2.5 字典树	7
2.6 平衡树	8
2.7 根号分治	9
2.7.1 无向图三元环计数	10
2.8 莫队	11
3 树	11
3.1 LCA	11
3.2 树的直径	12
3.3 树链剖分	13
3.4 树上启发式合并	14
4 图论	16
4.1 SPFA	16
4.2 Dijkstra	16
4.3 最小生成树	17
4.4 Kruskal 重构树	17
4.5 欧拉回路	18
4.6 二分图	19
4.6.1 二分图染色	19
4.6.2 二分图最大权匹配	19
4.7 差分约束	19
4.8 缩点	20
4.8.1 SCC(强连通分量)	20
4.8.2 DECC(边双联通分量)	21
5 字符串	22
5.1 字符串哈希	22
5.2 KMP	22
5.3 Manacher(马拉车)	22

6 动态规划	23
6.1 01 背包	23
6.2 完全背包	23
6.3 多重背包	23
6.4 树上背包	24
6.5 换根 dp	25
6.6 区间 dp	25
6.7 SOS dp	26
6.8 数位 dp	26
7 数学	27
7.1 筛法求因数个数	27
7.2 筛法求欧拉函数	27
7.3 筛法求莫比乌斯函数	27
7.4 矩阵快速幂	28
7.5 第二类斯特林数	28
7.6 卡特兰数	29
7.7 排列组合类-含 Lucas	29
7.8 高斯消元	29
7.9 Pollard-Rho	30
7.10 异或线性基	31
7.11 数论分块	31
7.12 莫比乌斯反演	32
7.13 <code>exgcd</code>	33
8 计算几何	33
8.1 极角排序	33
9 Other Trick	34
9.1 多路归并 (含二分解)	34
9.2 使用栈做递归防溢出	36
9.3 Log Trick	37
9.4 异或前缀模 4 小规律	38
9.5 曼哈顿距离与切比雪夫距离的相互转化	38
9.6 判定可重集合是否相等	39

1 杂项

1.1 IO 加速

Listing 1: 快速读入模板

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // ===== 通用快读 (支持任意整数类型) =====
5 template<typename T>
6 inline T read() {
7     T x = 0; int f = 1; char ch = getchar();
8     while (ch < '0' || ch > '9') { if (ch == '-') f = -1;
9     ch = getchar(); }
10    while (ch >= '0' && ch <= '9') { x = x * 10 + (ch - '0'); ch = getchar(); }
11    return x * f;
12 }
13
14 // ===== 通用快写 (支持任意整数类型) =====
15 template<typename T>
16 inline void write(T x) {
17     if (x < 0) { putchar('-'); x = -x; }
18     if (x > 9) write(x / 10);
19     putchar(x % 10 + '0');
20 }
21
22 // ===== 使用示例 =====
23 int main() {
24     int a = read<int>();
25     long long b = read<long long>();
26     short c = read<short>();
27     __int128 d = read<__int128>(); // 支持 __int128!
28
29     write(a); putchar('\n');
30     write(b); putchar(' ');
31     write(c); putchar('\n');
32     write(d); putchar('\n'); // __int128 直接输出完美
33     return 0;
34 }
```

1.2 __int128 操作

```
1 #include <iostream>
2 #include <cctype>
3 using namespace std;
4
5 // 重载 >> 运算符
6 istream& operator>>(istream &is, __int128 &n) {
7     n = 0;
8     int f = 1;
9     char ch = is.get();
10    while (!isdigit(ch)) {
11        if (ch == '-') f = -1;
12        ch = is.get();
13    }
14    while (isdigit(ch)) {
15        n = n * 10 + (ch - '0');
```

```
16        ch = is.get();
17    }
18    n *= f;
19    return is;
20 }
21
22 // 重载 << 运算符
23 ostream& operator<<(ostream &os, __int128 n) {
24     if (n < 0) {
25         os << '-';
26         n = -n;
27     }
28     if (n > 9) os << n / 10;
29     os << (char)(n % 10 + '0');
30     return os;
31 }
32
33 // 绝对值函数
34 __int128 abs128(__int128 x) {
35     return x < 0 ? -x : x;
36 }
37
38 // 整数平方根 (向下取整)
39 __int128 sqrt128(__int128 x) {
40     if (x < 0) return -1;
41     __int128 l = 0, r = 1;
42     while (r <= x / r) r *= 2; // 避免溢出
43     while (l < r) {
44         __int128 mid = (l + r + 1) / 2;
45         if (mid <= x / mid) l = mid;
46         else r = mid - 1;
47     }
48     return l;
49 }
50
51 int main() {
52     __int128 a, b;
53     cin >> a >> b;
54     cout << "a = " << a << ", b = " << b << endl;
55     cout << "abs(a) = " << abs128(a) << endl;
56     cout << "sqrt(b) = " << sqrt128(b) << endl;
57     return 0;
58 }
```

1.3 对拍

Listing 2: 对拍 windows

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //一直循环, 直到找到不一样的数据
5     while (1) {
6         system("data.exe > data.txt");
7         system("bf.exe < data.txt>bf.txt");
8         system("test.exe < data.txt > test.txt");
9         if (system("fc test.txt bf.txt")) break;
10        //不一样就跳出循环
11    }//当 fc 返回 1 时, 说明这时数据不一样
12    return 0;
13 }
```

Listing 3: 对拍 linux

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //一直循环，直到找到不一样的数据
5     while (1) {
6         system("./data > data.txt");
7         system("./bf < data.txt > bf.txt");
8         system("./test < data.txt > test.txt");
9         if (system("diff test.txt bf.txt")) break;
10        //不一样就跳出循环
11    }//当 diff 返回非0 时，说明这时数据不一样
12    return 0;
13 }

```

1.4 热身赛测试语句

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 void test_speed() {
5     long long cnt = 0;
6     auto start = chrono::high_resolution_clock::now();
7     for(int i = 0; i < 1000000000; ++i) cnt += i; // 10^9
8     // 次加法
9     auto end = chrono::high_resolution_clock::now();
10    chrono::duration<double> diff = end - start;
11    cout << "10^9 additions: " << diff.count() << "s" << endl;
12 }
13 volatile ll sink = 0; // 防止编译器优化：将结果写入
14 // volatile 变量
15 void test_time() {
16     const int batch = 100000; // 每批执行 10 万次运算，减少
17     // 时间检查开销
18     ll add_cnt = 0, mul_cnt = 0, mod_cnt = 0;
19
20     // ===== 1. 测试加法 =====
21     {
22         ll v = 0;
23         auto start = chrono::high_resolution_clock::now();
24
25         while (true) {
26             // 执行一批运算
27             for (int i = 0; i < batch; ++i) v += 1;
28             add_cnt += batch;
29
30             // 检查时间
31             auto now = chrono::high_resolution_clock::now();
32
33             if (chrono::duration_cast<chrono::seconds>(
34                 now - start).count() >= 1)
35                 break;
36         }
37         sink = v; // 确保 v 被使用，防止优化
38     }
39
40     // ===== 2. 测试乘法 =====
41     {
42         ll v = 1;
43         auto start = chrono::high_resolution_clock::now();
44
45         while (true) {
46             // 执行一批乘法
47             for (int i = 0; i < batch; ++i) v *= 2;
48             mul_cnt += batch;
49
50             // 检查时间
51             auto now = chrono::high_resolution_clock::now();
52
53             if (chrono::duration_cast<chrono::seconds>(
54                 now - start).count() >= 1)
55                 break;
56         }
57         sink = v; // 确保 v 被使用，防止优化
58     }
59
60     // ===== 3. 测试取模 =====
61     {
62         ll v = 0;
63         auto start = chrono::high_resolution_clock::now();
64
65         while (true) {
66             for (int i = 0; i < batch; ++i) v = (v + 1) % MOD;
67             mod_cnt += batch;
68
69             auto now = chrono::high_resolution_clock::now();
70
71             if (chrono::duration_cast<chrono::seconds>(
72                 now - start).count() >= 1)
73                 break;
74         }
75         sink = v;
76     }
77
78     cout << "1s add nums " << add_cnt << endl;
79     cout << "1s mul nums " << mul_cnt << endl;
80     cout << "1s mod&add nums " << mod_cnt << endl;
81 }
82 int main() {
83     // 1. 测试编译器版本
84     #ifdef __GNUC__
85         cout << "GCC Version: " << __GNUC__ << "." <<
86             __GNUC_MINOR__ << endl;
87     #endif
88     // 2. 测试 __int128
89     #ifdef __SIZEOF_INT128__
90         cout << "__int128 is supported" << endl;
91     #endif
92     test_speed(); // 3. 测试速度
93     test_time(); // 4. 测试各运算1秒内执行次数
94     return 0;
95 }

```

```

38 while (true) {
39     for (int i = 0; i < batch; ++i) {
40         v *= 2;
41         if (v > 1e18) v = 1; // 防止溢出
42     }
43     mul_cnt += batch;
44
45     auto now = chrono::high_resolution_clock::now();
46
47     if (chrono::duration_cast<chrono::seconds>(
48         now - start).count() >= 1)
49         break;
50     }
51     sink = v;
52 }
53
54 // ===== 3. 测试取模 =====
55 {
56     const ll MOD = 1e9+7;
57     ll v = 0;
58     auto start = chrono::high_resolution_clock::now();
59
60     while (true) {
61         for (int i = 0; i < batch; ++i) v = (v + 1) % MOD;
62         mod_cnt += batch;
63
64         auto now = chrono::high_resolution_clock::now();
65
66         if (chrono::duration_cast<chrono::seconds>(
67             now - start).count() >= 1)
68             break;
69     }
70     sink = v;
71 }
72
73 cout << "1s add nums " << add_cnt << endl;
74 cout << "1s mul nums " << mul_cnt << endl;
75 cout << "1s mod&add nums " << mod_cnt << endl;
76 }
77 int main() {
78     // 1. 测试编译器版本
79     #ifdef __GNUC__
80         cout << "GCC Version: " << __GNUC__ << "." <<
81             __GNUC_MINOR__ << endl;
82     #endif
83     // 2. 测试 __int128
84     #ifdef __SIZEOF_INT128__
85         cout << "__int128 is supported" << endl;
86     #endif
87     test_speed(); // 3. 测试速度
88     test_time(); // 4. 测试各运算1秒内执行次数
89     return 0;
90 }

```

Listing 4: 随机数

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 mt19937 rng; // 32 位梅森旋转生成器
4 mt19937_64 rng64; // 64 位梅森旋转生成器
5 struct Random {
6     // 构造函数：用随机设备 + 时间戳初始化种子
7     Random() {

```

```

8     auto seed = random_device{}();
9     auto time_seed = chrono::steady_clock::now().
time_since_epoch().count();
10    rng.seed(seed ^ time_seed);
11    rng64.seed(seed ^ time_seed);
12
13    }
14    template<typename T>
15    T randInt(T l, T r) {
16        //static_assert(is_integral<T>::value, "T must be an
integral type");
17        uniform_int_distribution<T> dist(l, r);
18        return dist(rng);
19    } // 生成 [l, r] 范围内的随机整数 (支持 int/long long/
short 等)
20    template<typename T>
21    T randDouble(T l, T r) {
22        //static_assert(is_floating_point<T>::value, "T must
be a floating point type");
23        uniform_real_distribution<T> dist(l, r);
24        return dist(rng);
25    } // 生成 [l, r] 范围内的随机浮点数 (支持 float/double/
long double)
26 };
27 // ----- 示例用法 -----
28 int main() {
29     Random rnd; // 全局或局部声明均可
30     int a = rnd.randInt(1, 100); // [1, 100]
31     long long b = rnd.randInt(0LL, 114514LL); // [0,
114514]
32     double c = rnd.randDouble(0.0, 1e18); // [0.0, 1e18]
33     std::vector<int> vec = {1, 2, 3, 4, 5};
34     shuffle(vec.begin(), vec.end(), rng);
35     cout<<"a="<<a<<endl;
36     cout<<"b="<<b<<endl;
37     cout<<fixed<<setprecision(10)<<c<<endl;
38     cout<<"vec:=";
39     for(int x:vec) cout<<x<<' ';
40     return 0;
41 }

```

1.6 cpp 扩栈 (win)

Listing 8: 扩栈指令

```

1 int size(256<<20);
2 __asm__ ( "movq %0, %%rsp\n"::"r"((char*)malloc(size)+
size));
3 /*
4 主程序
5 */
6 exit(0);

```

1.5 编译指令

Listing 5: 编译指令

```

1 g++ -std=c++20 -Wall -Wextra -O2 a.cpp -o a.exe && a.exe
//编译程序并运行
2 /*
3 -Wall: 开启大部分常用警告
4 -Wextra: 开启额外警告, 帮助发现潜在问题
5 -O2: 二级优化, 平衡编译速度与运行效率, 竞赛中最常用
6 -o a.exe: 指定输出文件名 (在 Linux 下可改为 -o a)
7 */

```

Listing 6: .bat/.cmd (win) 批处理文件编译

```

1 @echo off
2 g++ -std=c++17 -O2 -Wall -o sol.exe sol.cpp && sol.exe
3 pause

```

Listing 7: .sh (linux) 文件编译

```

1 #!/bin/bash
2 g++ -std=c++17 -O2 -Wall -o sol sol.cpp && ./sol

```

2 数据结构

2.1 ST 表

Listing 9: ST 表

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 const ll INF=4e18;
6 struct ST{
7     ST(){};
8     vector<vector<ll>>mx,mn;
9     ST(const vector<ll>&a){
10         int n=a.size()-1;
11         mx=vector<vector<ll>>(n+1,vector<ll>(21,-INF));
12         mn=vector<vector<ll>>(n+1,vector<ll>(21,INF));
13         for(int i=1;i<=n;i++) mx[i][0]=mn[i][0]=a[i];
14         for(int i=1;i<=21;i++){
15             for(int j=1;j+(1<<i)-1<=n;j++){
16                 mx[j][i]=(mx[j][i-1]>mx[j+(1<<i-1)][i-1])
17                 ?mx[j][i-1]:mx[j+(1<<i-1)][i-1];
18                 mn[j][i]=(mn[j][i-1]<mn[j+(1<<i-1)][i-1])
19                 ?mn[j][i-1]:mn[j+(1<<i-1)][i-1];
20             }
21         }
22     };
23     ll qry_mx(int l,int r){
24         int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
25         return (mx[l][len]>mx[r-(1<<len)+1][len])?mx[l][len]:mx[r-(1<<len)+1][len];
26     }
27     ll qry_mn(int l,int r){
28         int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
29         return (mn[l][len]<mn[r-(1<<len)+1][len])?mn[l][len]:mn[r-(1<<len)+1][len];
30     }
31 };
32 int main(){
33     cin.tie(0)->ios::sync_with_stdio(false);
34     int n,q,l,r;cin>>n>>q;
35     vector<ll>a(n+1);
36     for(int i=1;i<=n;i++) cin>>a[i];
37     ST st(a);
38     while(q--){
39         cin>>l>>r;
40         cout<<st.qry_mx(l,r)<<endl;
41     }
42     return 0;
43 }
```

2.2 并查集

Listing 10: 普通并查集

```
1 struct DSU{
2     vector<int>f;
3     vector<int>siz;
4     DSU(){};
5     DSU(int n){
6         f.resize(n+1);
```

```
7         iota(f.begin(),f.end(),0);
8         siz.assign(n+1,1);
9     }
10     int find(int x){
11         return x==f[x]?x:f[x]=find(f[x]);
12     }
13     bool merge(int x,int y){
14         int fx=find(x),fy=find(y);
15         if(fx==fy) return false;
16         f[fx]=fy;
17         siz[fy]+=siz[fx];
18         return true;
19     }
20     int nxt(int l,int r){
21         ll res=0;
22         l=find(l);
23         int fl=l;
24         while(l<=r){
25             ++res;
26             fl=find(l+1);
27             f[l]=fl;
28             l=fl;
29         }
30         return res;
31 }//l需要多少步跳到r? 模拟链表的模式删除l-r所有中转点
32 bool same(int x,int y){
33     return find(x)==find(y);
34 }
35 };
```

Listing 11: weighted-dsu

```

1 //P1196银河英雄传说
2 struct DSU{
3     DSU(){}
4     vector<int>f,siz;
5     vector<ll>dis;
6     DSU(int n){
7         dis.resize(n+1);
8         f.resize(n+1);
9         iota(f.begin(),f.end(),0);
10        siz.assign(n+1,1);
11    }
12    int find(int x){
13        int fx=f[x];
14        if(fx==x) return x;
15        int rt=find(f[x]);
16        dis[x]+=dis[f[x]];
17        return f[x]=rt;
18    }
19    bool merge(int x,int y){
20        int fx=find(x),fy=find(y);
21        if(fx==fy) return false;
22        f[fx]=fy;
23        dis[fx]+=siz[fy];
24        siz[fy]+=siz[fx];
25        return true;
26    }
27    bool same(int x,int y){
28        return find(x)==find(y);
29    }
30    ll Dis(int x,int y){
31        return dis[x]-dis[y];
32    }
33 };
34 void solve(){
35     int n;cin>>n;
36     DSU dsu(n+1);
37     for(int i=1;i<=n;i++){
38         char c;int x,y;cin>>c>>x>>y;
39         if(c=='M') dsu.merge(x,y);
40         else{
41             if(dsu.same(x,y)) cout<<abs(dsu.Dis(x,y))-1<<
endl;
42             else cout<<-1<<endl;
43         }
44     }
45 }

```

2.3 线段树 (普通区间和查询)

Listing 12: common-Seg

```

1 struct Seg{
2     #define lc (p<<1)
3     #define rc (p<<1|1)
4     struct S{
5         int l,r;
6         ll val,laz;
7     };
8     Seg(){};
9     Seg(int n){
10        tre.resize(n<<2);

```

```

11        build(0,n,1);
12    };
13    void pushup(int p){
14        tre[p].val=tre[lc].val+tre[rc].val;
15    }
16    void pushdown(int p){
17        if(tre[p].laz==0) return;
18        tre[lc].laz+=tre[p].laz;
19        tre[rc].laz+=tre[p].laz;
20        tre[lc].val+=tre[p].laz;
21        tre[rc].val+=tre[p].laz;
22        tre[p].laz=0;
23    }
24    void build(int l,int r,int p){
25        tre[p]={l,r,0,0};
26        if(l==r){
27            tre[p]=a[l];
28            return;
29        }
30        int mid=l+r>>1;
31        build(l,mid,lc);
32        build(mid+1,r,rc);
33        pushup(p);
34    }
35    void add(int l,int r,ll k,int p){
36        if(l<=tre[p].l&&r>=tre[p].r){
37            tre[p].val+=k*(tre[p].r-tre[p].l+1);
38            tre[p].laz+=k;
39            return;
40        }
41        int mid=tre[p].l+tre[p].r>>1;
42        pushdown(p);
43        if(l<=mid) add(l,r,k,lc);
44        if(r>mid) add(l,r,k,rc);
45        pushup(p);
46    }
47    ll qry(int l,int r,int p){
48        if(l<=tre[p].l&&r>=tre[p].r){
49            return tre[p].val;
50        }
51        int mid=tre[p].l+tre[p].r>>1;
52        pushdown(p);
53        ll ans=0;
54        if(l<=mid) ans+=qry(l,r,lc);
55        if(r>mid) ans+=qry(l,r,rc);
56        return ans;
57    }
58 };

```

2.4 笛卡尔树

Listing 13: cartesian-tree

```

1 //P7972 Self Permutation
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 const ll mod=1e9+7;
7 void solve(){
8     int n;cin>>n;
9     vector<int>L(n+1),R(n+1),st,a(n+1);

```

```

10 vector<ll>dp(n+1);
11 for(int i=1;i<=n;i++){
12     cin>>a[i];
13     int lc=0;
14     while(!st.empty()&&a[st.back()]>=a[i]){
15         lc=st.back();
16         st.pop_back();
17     }
18     L[i]=lc;
19     if(!st.empty()) R[st.back()]=i;
20     st.emplace_back(i);
21 }
22 auto dfs=[&](auto &&dfs,int u,int l,int r) ->void {
23     if(L[u]) dfs(dfs,L[u],l,u-1);
24     if(R[u]) dfs(dfs,R[u],u+1,r);
25     (dp[u]+=(dp[L[u]]+1)*(dp[R[u]]+1)%mod)%=mod;
26     if(l>1) (dp[u]+=dp[R[u]])%=mod;
27     if(r<n) (dp[u]+=dp[L[u]])%=mod;
28 };
29 dfs(dfs,st.front(),1,n);
30 cout<<dp[st.front()]<<endl;
31 }

```

2.5 字典树

Listing 14: Trie

```

1 struct Trie{
2     vector<int>cnt;
3     vector< array<int,65> >son;
4     int idx=0;
5     Trie(int n){
6         cnt.resize(n+1);
7         son.resize(n+1);
8     }
9     void insert(string &s){
10         int num;
11         int p=0;
12         for(char &c:s){
13             if(isdigit(c)) num=c-'0'+52;
14             else if(islower(c)) num=c-'a'+26;
15             else if(isupper(c)) num=c-'A';
16             if(!son[p][num]) son[p][num]=(++idx);
17             p=son[p][num];
18             ++cnt[p];
19         }
20     }
21     int query(string &s){
22         int num;
23         int p=0;
24         for(char &c:s){
25             if(isdigit(c)) num=c-'0'+52;
26             else if(islower(c)) num=c-'a'+26;
27             else if(isupper(c)) num=c-'A';
28             if(!son[p][num]) return 0;
29             p=son[p][num];
30         }
31         return cnt[p];
32     }
33 }trie;

```

Listing 15: 01-Trie

```

1 //P4551 最长异或路径
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 struct ZTrie{
7     vector<array<int,2>>son;
8     vector<int>cnt;
9     int idx=0;
10    ZTrie(int n){
11        son.resize(n+1);
12        cnt.resize(n+1);
13    }
14    void insert(int x){
15        int p=0;
16        for(int i=31;i>=0;i--) {
17            int bit=(x>>i&1);
18            if(!son[p][bit]) son[p][bit]=(++idx);
19            p=son[p][bit];
20        }
21    }
22    int query(int x){
23        int ans=0,p=0;
24        for(int i=31;i>=0;i--){
25            int bit=(x>>i&1);
26            if(son[p][bit^1]) p=son[p][bit^1],ans|=(1<<i);
27            ;
28            else if(son[p][bit]) p=son[p][bit];
29            else return ans;
30        }
31        return ans;
32    }
33 }trie(2e6);
34 const int N=2e5+5;
35 struct node{
36     int u,w;
37 };
38 vector<node>G[N];
39 int ans=0;
40 void dfs(int u,int fa,int sum){
41     trie.insert(sum);
42     ans=max(ans,trie.query(sum));
43     for(auto &[v,w]:G[u]){
44         if(v==fa) continue;
45         dfs(v,u,sum^w);
46     }
47 }
48 void solve(){
49     int n;cin>>n;
50     for(int i=1;i<=n;i++){
51         int x,y,w;cin>>x>>y>>w;
52         G[x].push_back({y,w});
53         G[y].push_back({x,w});
54     }
55     dfs(1,0,0);
56     cout<<ans<<endl;
57 }
58 int main(){
59     cin.tie(0)->ios::sync_with_stdio(false);
60     solve();
61     return 0;
62 }

```


2.6 平衡树

Listing 16: FHQ-Treap

```

1  template <typename T>
2  struct FHQ_Treap {
3      struct Node {
4          T data;
5          int l = 0, r = 0, sz = 1;
6          unsigned rnd;
7          bool rev = false; // 翻转懒标记
8      };
9      std::vector<Node> t; // 下标从1开始, 0为空节点
10     int root = 0;
11     FHQ_Treap() {
12         t.push_back(Node{}); // 占位0号
13         t[0].sz = 0;
14     }
15     // 完全清空
16     void clear() {
17         t.clear();
18         t.push_back(Node{});
19         t[0].sz = 0;
20         root = 0;
21     }
22     bool empty() const { return t[root].sz == 0; }
23     int size() const { return t[root].sz; }
24     void push(int u) {
25         if (!u || !t[u].rev) return;
26         swap(t[u].l, t[u].r);
27         if (t[u].l) t[t[u].l].rev ^= 1;
28         if (t[u].r) t[t[u].r].rev ^= 1;
29         t[u].rev = false;
30     }
31     // 新建节点 (随机优先级)
32     int new_node(const T& val) {
33         static std::mt19937 rng(std::chrono::steady_clock
34         ::now().time_since_epoch().count());
35         t.push_back({val, 0, 0, 1, static_cast<unsigned>(
36         rng())});
37         return static_cast<int>(t.size()) - 1;
38     }
39     void pull(int u) {
40         if (!u) return;
41         t[u].sz = 1 + t[t[u].l].sz + t[t[u].r].sz;
42         // 若T支持区间信息合并, 可在此调用 t[u].data.pull(t
43         [t[u].l].data, t[t[u].r].data);
44     }
45     // 按大小分裂: 左树得到前k个元素, 右树为剩余部分
46     void split(int u, int k, int& L, int& R) {
47         if (!u) { L = R = 0; return; }
48         push(u); // 关键!
49         int left_sz = t[t[u].l].sz;
50         if (k <= left_sz) {
51             R = u;
52             split(t[u].l, k, L, t[u].l);
53         } else {
54             L = u;
55             split(t[u].r, k - left_sz - 1, t[u].r, R);
56         }
57         pull(u);
58     }
59     // 合并两棵树 (L所有元素在R之前)
60     int merge(int L, int R) {
61         if (!L || !R) return L ? L : R;
62         push(L);
63         push(R); // 关键!
64         if (t[L].rnd < t[R].rnd) {
65             t[L].r = merge(t[L].r, R);
66             pull(L);
67             return L;
68         } else {
69             t[R].l = merge(L, t[R].l);
70             pull(R);
71             return R;
72         }
73     }
74     // ----- 位置操作 (1-indexed) -----
75     // 在第pos个元素之后插入 (pos = 0 表示头部, pos = size
76     () 表示尾部)
77     void insert(int pos, const T& val) {
78         int L, R;
79         split(root, pos, L, R);
80         int u = new_node(val);
81         root = merge(merge(L, u), R);
82     }
83     void reverse(int l, int r) { // 翻转 [l, r] (1-
84     indexed)
85         // assert(1 <= l && l <= r && r <= size());
86         int L, M, R;
87         split(root, l - 1, L, M);
88         split(M, r - l + 1, M, R);
89         t[M].rev ^= 1; // 打上翻转标记
90         root = merge(merge(L, M), R);
91     }
92     // 删除第k个元素 (1-indexed), 返回被删除的值
93     T erase(int k) {
94         int L, M, R;
95         split(root, k - 1, L, M);
96         split(M, 1, M, R);
97         T removed = t[M].data;
98         root = merge(L, R);
99         // 可选: 回收节点 M (本例中忽略, vector不回收)
100        return removed;
101    }
102    // 删除区间 [l, r] (1-indexed 闭区间)
103    void erase_range(int l, int r) {
104        int L, M, R;
105        split(root, l - 1, L, M);
106        split(M, r - l + 1, M, R);
107        root = merge(L, R);
108        // M 整棵子树被丢弃
109    }
110    // 查询第k个元素 (1-indexed)
111    T get_kth(int k) {
112        int u = root;
113        while (u) {
114            push(u); // 关键!
115            int left_sz = t[t[u].l].sz;
116            if (k <= left_sz) {
117                u = t[u].l;
118            } else if (k == left_sz + 1) {
119                return t[u].data;
120            } else {
121                k -= left_sz + 1;
122                u = t[u].r;
123            }
124        }
125    }

```

```

117         u = t[u].r;
118     }
119 }
120 return T{}; // 不会到达
121 }
122 // 便捷访问 (同 get_kth)
123 T operator[](int k) { return get_kth(k); }
124 // ----- 构建操作 -----
125 // 线性时间构建 (笛卡尔树) —— 输入序列的顺序即为最终中
序顺序
126 void build(const std::vector<T>& arr) {
127     clear(); // 清空原有树
128     if (arr.empty()) return;
129     std::vector<int> nodes; // 存储每个节点在 t
中的下标
130     nodes.reserve(arr.size());
131     for (const auto& val : arr) {
132         nodes.push_back(new_node(val));
133     }
134     // 单调栈维护右链, 确保按随机优先级满足堆性质
135     std::vector<int> St;
136     for (int cur : nodes) {
137         int last = 0;
138         while (!St.empty() && t[St.back()].rnd > t[
cur].rnd) {
139             last = St.back();
140             St.pop_back();
141         }
142         t[cur].l = last; // 当前节点的左
子 = 最后一个被弹出的节点
143         if (!St.empty()) {
144             t[St.back()].r = cur; // 栈顶的右子指向
当前节点
145         }
146         St.push_back(cur);
147     }
148     // 栈底即为根
149     root = St.empty() ? 0 : St[0];
150     // 后序遍历更新所有节点的 sz (因为只设置了左右关系,
sz 仍为1)
151     update_subtree_size(root);
152 }
153 private:
154     // 后序遍历更新子树大小
155     void update_subtree_size(int u) {
156         if (!u) return;
157         update_subtree_size(t[u].l);
158         update_subtree_size(t[u].r);
159         pull(u);
160     }
161 };

```

2.7 根号分治

- 第一行: 两个整数 n, q ($1 \leq n, q \leq 10^5$), 分别表示序列长度和询问次数。
- 第二行: n 个整数 $A_1, A_2, \dots, A_n$ ($1 \leq A_i \leq 10^9$), 表示给定的整数序列。
- 接下来 q 行: 每行两个整数 x, y ($1 \leq x, y \leq 10^9$),

表示一次询问。

输出格式

输出 q 行, 每行一个整数, 表示对应询问的答案。

简要分析

给定序列 $A[1..n]$ 和询问 (x, y) , 需统计满足 $i < j$ 且 $A[i] = x, A[j] = y$ 的数对数量。记 $pos[v]$ 为值 v 在序列中出现的位置集合 (升序排列), 则答案可表示为:

$$\sum_{p \in pos[x]} (|pos[y]| - \text{upper_bound}(pos[y], p))$$

或等价表示为:

$$\sum_{p \in pos[y]} \text{lower_bound}(pos[x], p)$$

核心划分

设阈值 $B = \sqrt{n}$ (取整), 将所有值分为两类:

1. 高频值 (Heavy): 满足 $|pos[v]| > B$ 的值, 记高频值集合为 H ;
2. 低频值 (Light): 满足 $|pos[v]| \leq B$ 的值。

预处理阶段 (离线预处理高频值对)

对于任意两个高频值 $u, v \in H$, 预计算数对 (u, v) 的答案并存储在哈希表 mp 中:

$$mp[(u, v)] = \sum_{p \in pos[u]} (|pos[v]| - \text{upper_bound}(pos[v], p))$$

预处理时间复杂度: $O(|H|^2 \cdot B)$, 由于 $|H| \leq \frac{n}{B}$, 故总复杂度为 $O(nB)$ 。

查询阶段 (分情况处理)

对每个询问 (x, y) , 分三类处理 (结合记忆化避免重复计算):

1. 若 (x, y) 已计算过 (记忆化哈希表 memo 存在), 直接返回结果;
2. 若 x, y 均为高频值, 直接调用预处理结果 $mp[(x, y)]$;
3. 若至少一个为低频值, 选择更小的集合遍历计算:

- 若 $|pos[x]| \leq |pos[y]|$: 遍历 $pos[x]$, 对每个位置 p 统计 $pos[y]$ 中大于 p 的元素数量;
- 否则: 遍历 $pos[y]$, 对每个位置 p 统计 $pos[x]$ 中小于 p 的元素数量。

单次查询时间复杂度: $O(B)$ 。

复杂度分析

预处理复杂度: $O(n\sqrt{n})$; 单次查询复杂度: $O(\sqrt{n})$, 总查询复杂度: $O(q\sqrt{n})$; 整体时间复杂度: $O((n+q)\sqrt{n})$, 可通过 10^5 规模的数据。

Listing 17: 根号分治

```

1 void solve(){
2     int n,q;cin>>n>>q;
3     vector<int>a(n+1);
4     unordered_map< int,vector<int> >pos;//存储重数
5     for(int i=1;i<=n;i++){
6         cin>>a[i];
7         pos[a[i]].push_back(i);
8     }
9     int siz=sqrt(n+0.5);
10    vector<int>H;
11    for(auto const&[x,y]:pos){
12        if(y.size()>siz) H.emplace_back(x);
13    }
14    map<pair<int,int>,ll>mp;
15    for(int i=0;i<H.size();i++){
16        for(int j=0;j<H.size();j++){
17            for(int &p:pos[H[i]]){
18                for(int &p:pos[H[j]]){
19                    mp[{H[i],H[j]}]+=pos[H[j]].end()-
20                    upper_bound(pos[H[j]].begin(),pos[H[j]].end(),p);
21                }
22            }
23        }
24    }
25    map<pair<int,int>,ll>memo;
26    for(int i=1;i<=q;i++){
27        int x,y;cin>>x>>y;
28        if(memo.count({x,y})){
29            cout<<memo[{x,y}]<<endl;
30            continue;
31        }
32        if(pos[x].size()>siz&&pos[y].size()>siz){
33            cout<<(memo[{x,y}]=mp[{x,y}])<<endl;
34        }
35        else{
36            ll ans=0;
37            if(pos[x].size()<=pos[y].size()||pos[x].size()<=siz&&pos[y].size()<=siz){
38                for(int &p:pos[x]){
39                    ans+=pos[y].end()-upper_bound(pos[y].begin(),pos[y].end(),p);
40                }
41            }
42            else{
43                for(int &p:pos[y]){
44                    ans+=pos[x].end()-upper_bound(pos[x].begin(),pos[x].end(),p);
45                }
46            }
47            memo[{x,y}]=ans;
48        }
49    }
50 }

```

```

42         ans+=lower_bound(pos[x].begin(),pos[x].end(),p)-pos[x].begin();
43     }
44 }
45     cout<<(memo[{x,y}]=ans)<<endl;
46 }
47 }
48 }

```

2.7.1 无向图三元环计数

题面简述

给定一个 n 个点 m 条边的简单无向图（无重边、无自环，不保证连通），求图中三元环的个数。

三元环指恰好包含三个点 u, v, w 及三条边 $\langle u, v \rangle, \langle v, w \rangle, \langle w, u \rangle$ 的子图。两个三元环不同当且仅当存在一个点属于其中一个而不属于另一个。

数据范围: $1 \leq n \leq 10^5$, $1 \leq m \leq 2 \times 10^5$ 。

做法

采用经典的无向边定向 + 枚举算法。

1. **定向**: 对每条无向边 (u, v) , 令其从度数小的点指向度数大的点; 若度数相同, 则从编号小的点指向编号大的点。这样将无向图转化为一个 DAG, 记有向边集为 E' 。
2. **枚举**: 对每个点 u , 先用标记数组标记 u 的所有出邻居。然后对每条出边 $u \rightarrow v$, 枚举 v 的所有出邻居 w , 若 w 被 u 标记过 (即 $u \rightarrow w \in E'$), 则 (u, v, w) 构成一个三元环, 答案加一。

由于定向后每个三元环的三条边恰好形成一条有向路径 $u \rightarrow v \rightarrow w$ (其中 $u \rightarrow w$ 也存在), 因此每个三元环恰好被统计一次, 无需去重。

复杂度分析

引理: 定向后每个点的出度 $d^+(u) \leq \sqrt{2m}$ 。

证明. 对任意点 u , 其每个出邻居 v 都满足 $\deg(v) \geq \deg(u) \geq d^+(u)$ 。因此 u 的 $d^+(u)$ 个出邻居的度数之和满足:

$$\sum_{v \in N^+(u)} \deg(v) \geq (d^+(u))^2$$

又因为所有边的度数贡献总和不超过 $2m$, 故 $(d^+(u))^2 \leq 2m$, 即 $d^+(u) \leq \sqrt{2m}$ 。□

算法的核心操作量为：

$$T = \sum_{(u \rightarrow v) \in E'} d^+(v) \leq \sum_{(u \rightarrow v) \in E'} \sqrt{2m} = |E'| \cdot \sqrt{2m} = m\sqrt{2m}$$

因此总时间复杂度为 $O(m\sqrt{m})$ 。

对于本题 $m = 2 \times 10^5$ ，操作量约为 $2 \times 10^5 \times 632 \approx 1.26 \times 10^8$ ，可以通过。空间复杂度为 $O(n+m)$ 。

Listing 18: 根号分治求三元环计数

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     cin.tie(nullptr)->ios::sync_with_stdio(false);
5     int n, m;
6     cin >> n >> m;
7     vector<pair<int,int>> edges(m);
8     vector<int>deg(n + 1, 0);
9     for(int i = 0; i < m; i++){
10         cin >> edges[i].first >> edges[i].second;
11         deg[edges[i].first]++;
12         deg[edges[i].second]++;
13     }
14     // 定向: 度数小 -> 度数大, 度数相同则编号小 -> 编号大
15     vector<vector<int>> adj(n + 1);
16     for(int i = 0; i < m; i++){
17         int u = edges[i].first, v = edges[i].second;
18         if(deg[u] < deg[v] || (deg[u] == deg[v] && u < v))
19     ){
20         adj[u].push_back(v);
21     } else {
22         adj[v].push_back(u);
23     }
24     }
25     // 枚举三元环
26     vector<int> mark(n + 1, 0); // mark[w] = u 表示 u->w
27     // 这条边存在
28     long long ans = 0;
29     for(int u = 1; u <= n; u++){
30         // 标记 u 的所有出边邻居
31         for(int v : adj[u]){
32             mark[v] = u;
33         }
34         // 枚举 u -> v -> w, 检查 u -> w
35         for(int v : adj[u]){
36             for(int w : adj[v]){
37                 if(mark[w] == u){
38                     ans++;
39                 }
40             }
41         }
42     }
43     cout << ans << "\n";
44     return 0;
45 }
```

2.8 莫队

Listing 19: 普通莫队算法

```
1 //P2709 小B的询问
```

```
2 const int N=5e4+10;
3 int a[N],pos[N],cnt[N];
4 ll res,ans[N];
5 struct Q{
6     int l,r,k;
7 }q[N];
8 void Add(int n){
9     cnt[a[n]]++;
10    res+=cnt[a[n]]*cnt[a[n]]-(cnt[a[n]]-1)*(cnt[a[n]]-1);
11 }
12 void Sub(int n){
13     cnt[a[n]]--;
14    res-=(cnt[a[n]]+1)*(cnt[a[n]]+1)-(cnt[a[n]])*(cnt[a[n]]);
15 }
16 int main(){
17     ios::sync_with_stdio(false);
18     cin.tie(0),cout.tie(0);
19     int n,m,k;cin>>n>>m>>k;
20     int siz=sqrt(n+0.5);
21     for(int i=1;i<=n;i++){
22         cin>>a[i];
23         pos[i]=i/siz;
24     }
25     for(int i=0;i<m;i++){
26         cin>>q[i].l>>q[i].r;
27         q[i].k=i;
28     }
29     sort(q,q+m,[](Q x,Q y){
30         return pos[x.l]==pos[y.l]?x.r<y.r:pos[x.l]<pos[y.l];
31     });
32     int l=1,r=0;
33     for(int i=0;i<m;i++){
34         while(q[i].l<l) Add(--l);
35         while(q[i].r>r) Add(++r);
36         while(q[i].l>l) Sub(l--);
37         while(q[i].r<r) Sub(r--);
38         ans[q[i].k]=res;
39     }
40     for(int i=0;i<m;i++){
41         cout<<ans[i]<<endl;
42     }
43     return 0;
44 }
```

3 树

3.1 LCA

Listing 20: dfn 大跳 LCA

```
1 //P3379 最近公共祖先
2 const int N=5e5+5;
3 vector<int>G[N];
4 int f[N],d[N],dfn[N],top[N],cnt,siz[N],son[N];
5 void dfs1(int u,int fa){
6     d[u]=d[fa]+1;
7     f[u]=fa;
8     siz[u]=1;
9     for(int v:G[u]){
10         if(fa==v) continue;
```

```

11     dfs1(v,u);
12     siz[u]+=siz[v];
13     if(siz[v]>siz[son[u]]) son[u]=v;
14 }
15 }
16 void dfs2(int u,int node){
17     dfn[u]=(++cnt); //维护dfn序
18     top[u]=node;
19     if(son[u]) dfs2(son[u],node);
20     for(int v:G[u]){
21         if(v==f[u]||v==son[u]) continue;
22         dfs2(v,v);
23     }
24 }
25 int LCA(int l,int r){
26     while(top[l]!=top[r]){
27         if(d[top[l]]<d[top[r]]) swap(l,r);
28         l=f[top[l]];
29     }
30     if(d[l]<d[r]) return l;
31     return r;
32 } //树上LCA跳
33 void solve(){
34     int n,m,s;
35     cin>>n>>m>>s;
36     for(int i=1;i<n;i++){
37         int x,y;cin>>x>>y;
38         G[x].push_back(y);
39         G[y].push_back(x);
40     }
41     dfs1(s,0);
42     dfs2(s,s);
43     for(int i=1;i<=m;i++){
44         int x,y;cin>>x>>y;
45         cout<<LCA(x,y)<<endl;
46     }
47 }

```

Listing 21: dfn 实现 O1 LCA

```

1 //P3379 最近公共祖先
2 void solve(){
3     int n,q,s;
4     cin>>n>>q>>s;
5     vector<vector<int>>G(n+1);
6     for(int i=1;i<n;i++){
7         int x,y;
8         cin>>x>>y;
9         G[x].push_back(y);
10        G[y].push_back(x);
11    }
12    vector<int>d(n+1);
13    vector<int>f(n+1);
14    vector<int>dfn(n+1);
15    int cnt=0;
16    auto dfs=[&](auto &&dfs,int u,int fa) ->void {
17        f[u]=fa;
18        dfn[u]=(++cnt);
19        d[u]=d[fa]+1;
20        for(int v:G[u]){
21            if(v==fa) continue;
22            dfs(dfs,v,u);
23        }

```

```

24    };
25    dfs(dfs,s,0);
26    vector<vector<int>>st(n+1,vector<int>(21));
27    for(int i=1;i<=n;i++) st[dfn[i]][0]=f[i];
28    for(int j=1;j<=20;j++){
29        for(int i=1;i+(1<<j)-1<=n;i++){
30            int l=st[i][j-1];
31            int r=st[i+(1<<j-1)][j-1];
32            if(d[l]<d[r]) st[i][j]=l;
33            else st[i][j]=r;
34        }
35    }
36    auto lca=[&](int l,int r) ->int {
37        if(l==r) return l;
38        l=dfn[l],r=dfn[r];
39        if(l>r) swap(l,r);
40        l++;
41        int len=__lg(r-l+1);
42        int L=st[l][len];
43        int R=st[r-(1<<len)+1][len];
44        return (d[L]<d[R])?L:R;
45    };
46    while(q--){
47        int x,y;
48        cin>>x>>y;
49        cout<<lca(x,y)<<endl;
50    }
51 }

```

3.2 树的直径

Listing 22: 两遍 dfs 查直径

```

1 const int N=2e5+10;
2 vector<int>G[N];
3 bool vis[N];
4 ll d[N],s=1;
5 void dfs(int u,int f){
6     for(auto &v:G[u]){
7         if(v==f) continue;
8         d[v]=d[u]+1;
9         dfs(v,u);
10    }
11 }
12 int main(){
13     cin.tie(0)->ios::sync_with_stdio(false);
14     int n;cin>>n;
15     for(int i=1,x,y;i<n;i++){
16         cin>>x>>y;
17         G[x].push_back(y);
18         G[y].push_back(x);
19     }
20     d[s]=0;dfs(s,0);
21     for(int i=1;i<=n;i++) if(d[i]>d[s]) s=i;
22     d[s]=0;dfs(s,0);
23     ll ans=0;
24     for(int i=1;i<=n;i++) ans=max(ans,d[i]);
25     cout<<ans<<endl;
26     return 0;
27 }

```

Listing 23: 树形 dp 求直径

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 int main(){
6     cin.tie(0)->ios::sync_with_stdio(false);
7     int n;cin>>n;
8     vector<vector<int>>>G(n+1);
9     vector<int>down(n+1),up(n+1);
10    for(int i=1;i<n;i++){
11        int x,y;cin>>x>>y;
12        G[x].emplace_back(y);
13        G[y].emplace_back(x);
14    }
15    auto dfs1=[&](auto &&dfs1,int u,int fa) ->void {
16        for(int v:G[u]){
17            if(v==fa) continue;
18            dfs1(dfs1,v,u);
19            down[u]=max(down[u],down[v]+1);
20        }
21    };
22    dfs1(dfs1,1,0);
23    auto dfs2=[&](auto &&dfs2,int u,int fa) ->void {
24        int m1=0,m2=0;//最长路径, 次长路径
25        for(int v:G[u]){
26            if(v==fa) continue;
27            if(down[m1]<down[v]) m2=m1,m1=v;
28            else if(down[m2]<down[v]) m2=v;
29        }
30        for(int v:G[u]){
31            if(v==fa) continue;
32            up[v]=up[u]+1;
33            if(m1!=0&&down[m1]!=down[v]) up[v]=max(up[v],
34            down[m1]+2);
35            else if(down[m2]!=0) up[v]=max(up[v],down[m2]
36            ]+2);
37            dfs2(dfs2,v,u);
38        }
39    };
40    dfs2(dfs2,1,0);
41    int ans=0;
42    for(int i=1;i<n;i++) ans=max(ans,down[i]+up[i]);
43    cout<<ans<<endl;
44    return 0;
45 }

```

3.3 树链剖分

Listing 24: 树链剖分

```

1 //P3384 [模板]重链剖分 / 树链剖分
2 const int N=1e5+5;
3 ll mod;
4 int n,m,r;
5 vector<int>G[N];
6 int d[N],f[N],siz[N],dfn[N],rev[N],cnt,top[N],son[N];
7 ll val[N],new_val[N];
8 struct Seg{
9     struct S{
10         int l,r;
11         ll sum,laz;
12     };

```

```

13     int n;
14     vector<S>tre;
15     Seg(){};
16     Seg(int n):n(n){
17         tre.resize(n<2);
18         build(0,n,1);
19     };
20     #define lc (p<<1)
21     #define rc (p<<1|1)
22     void pushup(int p){
23         tre[p].sum=(tre[lc].sum+tre[rc].sum)%mod;
24     }
25     void pushdown(int p){
26         if(tre[p].laz==0) return;
27         tre[lc].laz=(tre[lc].laz+tre[p].laz)%mod;
28         tre[rc].laz=(tre[rc].laz+tre[p].laz)%mod;
29         tre[lc].sum=(tre[lc].sum+tre[p].laz*(tre[lc].r-
30         tre[lc].l+1)%mod)%mod;
31         tre[rc].sum=(tre[rc].sum+tre[p].laz*(tre[rc].r-
32         tre[rc].l+1)%mod)%mod;
33         tre[p].laz=0;
34     }
35     void build(int l,int r,int p){
36         tre[p]={l,r,0LL,0LL};
37         if(l==r) {
38             tre[p].sum=new_val[l]%mod;
39             return;
40         }
41         int mid=l+r>>1;
42         build(l,mid,lc);
43         build(mid+1,r,rc);
44         pushup(p);
45     }
46     void Add(int l,int r,ll k,int p){
47         if(l<=tre[p].l&&r>=tre[p].r){
48             tre[p].sum=(tre[p].sum+k*(tre[p].r-tre[p].l
49             +1)%mod)%mod;
50             tre[p].laz=(tre[p].laz+k)%mod;
51             return;
52         }
53         int mid=tre[p].l+tre[p].r>>1;
54         pushdown(p);
55         if(l<=mid) Add(l,r,k,lc);
56         if(r>mid) Add(l,r,k,rc);
57         pushup(p);
58     }
59     ll qry(int l,int r,int p){
60         if(l<=tre[p].l&&r>=tre[p].r) return tre[p].sum;
61         ll ans=0;
62         int mid=tre[p].l+tre[p].r>>1;
63         pushdown(p);
64         if(l<=mid) ans=(ans+qry(l,r,lc))%mod;
65         if(r>mid) ans=(ans+qry(l,r,rc))%mod;
66         return ans;
67     }
68 }S;
69 void dfs1(int u,int fa){
70     f[u]=fa;
71     d[u]=d[fa]+1;
72     siz[u]=1;
73     for(int v:G[u]){
74         if(v==fa) continue;

```



```

72     dfs1(v,u);
73     siz[u]+=siz[v];
74     if(siz[v]>siz[son[u]]) son[u]=v;
75 }
76 }//第一遍初始化节点信息
77 void dfs2(int u,int node){
78     top[u]=node;
79     dfn[u]=(++cnt);
80     rev[cnt]=u;
81     new_val[cnt]=val[u];
82     if(son[u]) dfs2(son[u],node);
83     for(int v:G[u]){
84         if(v!=son[u]&&v!=f[u]) dfs2(v,v);
85     }
86 }
87 void Upd_path(int l,int r,ll k){
88     k%=mod;
89     while(top[l]!=top[r]){
90         if(d[top[l]]<d[top[r]]) swap(l,r);
91         S.Add(dfn[top[l]],dfn[l],k,1);
92         l=f[top[l]];
93     }
94     if(d[l]>d[r]) swap(l,r);
95     S.Add(dfn[l],dfn[r],k,1);
96 }
97 ll qry_path(int l,int r){
98     int res=0;
99     while(top[l]!=top[r]){
100         if(d[top[l]]<d[top[r]]) swap(l,r);
101         res=(res+S.qry(dfn[top[l]],dfn[l],1))%mod;
102         l=f[top[l]];
103     }//不在同一个重链中LCA跳
104     if(d[l]>d[r]) swap(l,r);
105     res=(res+S.qry(dfn[l],dfn[r],1))%mod;
106     return res;
107 }
108 void Upd(int u,ll k){
109     k%=mod;
110     S.Add(dfn[u],dfn[u]+siz[u]-1,k,1);
111 }
112 ll qry(int u){
113     return S.qry(dfn[u],dfn[u]+siz[u]-1,1);
114 }
115 void solve(){
116     cin>>n>>m>>r>>mod;
117     for(int i=1;i<=n;i++) cin>>val[i];
118     for(int i=1;i<=n;i++){
119         int u,v;cin>>u>>v;
120         G[u].push_back(v);
121         G[v].push_back(u);
122     }
123     dfs1(r,0);
124     dfs2(r,r);
125     S=Seg(n+1);
126     for(int i=1;i<=m;i++){
127         int op,l,r;ll k;cin>>op;
128         if(op==1){
129             cin>>l>>r>>k;
130             Upd_path(l,r,k);
131         }
132         else if(op==2){
133             cin>>l>>r;

```

```

134         cout<<qry_path(l,r)<<endl;
135     }
136     else if(op==3){
137         cin>>l>>k;
138         Upd(l,k);
139     }
140     else if(op==4){
141         cin>>l;
142         cout<<qry(l)<<endl;
143     }
144 }
145 }

```

3.4 树上启发式合并

题面简述

给一棵以 1 为根的树，共 n 个结点，每个结点有一个颜色 $c[i]$ 。每次询问给出一个结点 u ，求以 u 为根的子树中有多少种不同的颜色。

第一步：求 DFS 序与子树区间

对树进行一次 DFS，记录每个结点的进入时间 $in[u]$ 和离开时间 $out[u]$ 。则以 u 为根的子树恰好对应 DFS 序中的连续区间 $[in[u], out[u]]$ 。

同时在 DFS 过程中记录每个结点的重儿子（子树大小最大的儿子），记为 $heavy[u]$ 。

第二步：启发式合并（DSU on Tree）

对每个结点 u ，按如下顺序处理：

1. 递归处理所有轻儿子，处理完后清除其对全局计数数组的贡献。
2. 递归处理重儿子，处理完后保留其对全局计数数组的贡献。
3. 暴力遍历所有轻儿子的子树中的结点，将它们的颜色加入全局计数数组 $cnt[\cdot]$ ：

$$cnt[c[v]] \leftarrow cnt[c[v]] + 1$$

若某颜色 $c[v]$ 的计数从 0 变为 1，则当前不同颜色种类数加一：

$$tot \leftarrow tot + [cnt[c[v]] = 1]$$

4. 将结点 u 自身的颜色也加入计数数组，同理更新 tot 。
5. 此时 tot 即为以 u 为根的子树的颜色种类数，记录答案：

$$ans[u] = tot$$

6. 若 u 本身是其父结点的轻儿子，则暴力清除整棵子树的贡献（将 $\text{cnt}[\cdot]$ 和 tot 恢复）。

第三步：回答询问

所有询问离线处理。对于每个询问的结点 u ，直接输出 $\text{ans}[u]$ 即可。

正确性说明

启发式合并的核心思想在于：每次只保留重儿子的信息，轻儿子的信息暴力重算。由于每个结点到根的路径上至多有 $O(\log n)$ 条轻边（每经过一条轻边，子树大小至少翻倍），所以每个结点被暴力遍历的总次数为 $O(\log n)$ 。

复杂度分析

- **时间复杂度：**每个结点被暴力加入/清除计数数组的次数为其到根路径上的轻边数，至多 $O(\log n)$ 次。因此总时间复杂度为：

$$O(n \log n + m)$$

其中 m 为询问数。

Listing 25: 树上启发式合并

```
1 //U41492 树上数颜色
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 const int N=1e5+5;
7 vector<int>G[N];
8 int cnt[N],siz[N],son[N],f[N],res,a[N],ans[N];
9 void dfs1(int u,int fa){
10     f[u]=fa;
11     siz[u]=1;
12     for(int v:G[u]){
13         if(v==fa) continue;
14         dfs1(v,u);
15         siz[u]+=siz[v];
16         if(siz[v]>siz[son[u]]) son[u]=v;
17     }
18 }
19 void Upd(int u,bool F,int skipped){
20     if(F){
21         cnt[a[u]]++;
22         if(cnt[a[u]]==1) res++;
23     }
24     else{
25         cnt[a[u]]--;
26         if(cnt[a[u]]==0) res--;
27     }
28     for(int v:G[u]){
29         if(v==f[u] || v==skipped) continue;
```

```
30     Upd(v,F,0);
31 }
32 }//更新轻儿子贡献
33 void dfs2(int u,bool F){
34     for(int v:G[u]){
35         if(v==son[u] || v==f[u]) continue;
36         dfs2(v,0);
37     }
38     if(son[u]) {
39         dfs2(son[u],1);
40     }
41     Upd(u,1,son[u]);
42     ans[u]=res;
43     if(!F) Upd(u,0,0);
44 }
45 void solve(){
46     int n;cin>>n;
47     for(int i=1;i<=n;i++){
48         int x,y;cin>>x>>y;
49         G[x].push_back(y);
50         G[y].push_back(x);
51     }
52     for(int i=1;i<=n;i++) cin>>a[i];
53     dfs1(1,0);
54     dfs2(1,1);
55     int q;cin>>q;
56     while(q--){
57         int x;cin>>x;
58         cout<<ans[x]<<endl;
59     }
60 }
61 int main(){
62     cin.tie(0)->ios::sync_with_stdio(false);
63     solve();
64     return 0;
65 }
```


4 图论

4.1 SPFA

Listing 26: SPFA

```
1 struct node{
2     int u,ll w;
3 };
4 vector<node>G[N];
5 ll dis[N];
6 bool inq[N];
7 int n,m,s;
8 void spfa(int s){
9     fill(dis+1,dis+1+n,INT_MAX);
10    queue<int>Q;
11    Q.push(s);
12    dis[s]=0;
13    inq[s]=1;
14    while(!Q.empty()){
15        int u=Q.front();
16        Q.pop();
17        inq[u]=0;
18        for(auto &[v,w]:G[u]){
19            if(dis[v]>dis[u]+w){
20                dis[v]=dis[u]+w;
21                if(!inq[v]){
22                    Q.push(v);
23                    inq[v]=1;
24                }
25            }
26        }
27    }
28 }
29 void solve(){
30     cin>>n>>m>>s;
31     for(int i=1;i<=m;i++){
32         int x,y,ll w;
33         cin>>x>>y>>w;
34         G[x].push_back({y,w});
35     }
36     spfa(s);
37     for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
38     cout<<endl;
39 }
```

4.2 Dijkstra

Listing 27: Dijkstra(稀疏图堆优化版本)

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 const int N=2e5+5;
6 struct node{
7     int u,ll w;
8     friend bool operator<(node a,node b){
9         return a.w>b.w;
10    }
11 };
12 vector<node>G[N];
```

```
13 bool vis[N];
14 ll dis[N];
15 int n,m,s;
16 void Dijkstra(int s){
17     fill(dis+1,dis+1+n,INT_MAX);
18     priority_queue<node>Q;
19     dis[s]=0;
20     Q.push({s,0});
21     while(!Q.empty()){
22         int u=Q.top().u;
23         Q.pop();
24         if(vis[u]) continue;
25         vis[u]=1;
26         for(auto &[v,w]:G[u]){
27             if(dis[v]>dis[u]+w){
28                 dis[v]=dis[u]+w;
29                 Q.push({v,dis[v]});
30             }
31         }
32     }
33 }
34 void solve(){
35     cin>>n>>m>>s;
36     for(int i=1;i<=m;i++){
37         int x,y,ll w;
38         cin>>x>>y>>w;
39         G[x].push_back({y,w});
40     }
41     Dijkstra(s);
42     for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
43     cout<<endl;
44 }
45 int main(){
46     solve();
47     return 0;
48 }
```

Listing 28: Dijkstra(稠密图邻接矩阵版)

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 const int N=1010; // 注意:n<=1000 推荐使用稠密版, 否则
6 // 改用稀疏图堆优化!
7 const ll INF = 1e18;
8 ll g[N][N];
9 ll dis[N];
10 bool vis[N];
11 int n,m,s;
12 void Dijkstra(int s){
13     for(int i=1;i<=n;i++){
14         dis[i]=INF;
15         vis[i]=false;
16     }
17     dis[s]=0;
18     for(int i=1;i<=n;i++){
19         int u=-1; ll minv=INF;
20         for(int j=1;j<=n;j++){
21             if(!vis[j] && dis[j]<minv){
22                 minv=dis[j];
23             }
24         }
25         u=minv;
26         vis[u]=true;
27         for(int j=1;j<=n;j++){
28             if(dis[j]>dis[u]+g[u][j]){
29                 dis[j]=dis[u]+g[u][j];
30             }
31         }
32     }
33 }
```

```

24         u=j;
25     }
26 }
27 if(u==1) break;
28 vis[u]=true;
29 for(int j=1;j<=n;j++){
30     if(g[u][j]<INF && dis[j]>dis[u]+g[u][j]){
31         dis[j]=dis[u]+g[u][j];
32     }
33 }
34 }
35 }
36
37 void solve(){
38     cin>>n>>m>>s;
39     for(int i=1;i<=n;i++){
40         for(int j=1;j<=n;j++){
41             g[i][j]=(i==j?0:INF);
42         }
43         for(int i=1;i<=m;i++){
44             int x,y,ll w;
45             cin>>x>>y>>w;
46             g[x][y]=min(g[x][y],w); // 支持重边取最优
47         }
48         Dijkstra(s);
49         for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
50         cout<<endl;
51     }
52 }
53 int main(){
54     solve();
55     return 0;
56 }

```

4.3 最小生成树

Listing 29: Kruskal 求最小生成树

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 ll ans;int n,m,cntn;
6 const int N=2e5+10;
7 int f[N];
8 int find(int x){
9     if(x==f[x]) return x;
10    return f[x]=find(f[x]);
11 }
12 struct node{
13     int x,y,w;
14     friend bool operator <(node a,node b){
15         return a.w<b.w;
16     }
17 }a[N];
18 void kruskal(){
19     cntn=n;
20     for(int i=1;i<=m;i++){
21         int fx=find(a[i].x);
22         int fy=find(a[i].y);
23         if(fx!=fy){
24             f[fx]=fy;
25             cntn--;
26             ans+=a[i].w;

```

```

27         if(cntn==1) break;
28     }
29 }
30 }
31 int main(){
32     cin.tie(0)->ios::sync_with_stdio(false);
33     cin>>n>>m;
34     for(int i=1;i<=n;i++) f[i]=i;
35     for(int i=1;i<=m;i++){
36         int x,y,w;
37         cin>>a[i].x>>a[i].y>>a[i].w;
38     }
39     sort(a+1,a+1+m);
40     kruskal();
41     if(cntn==1) cout<<ans<<endl;
42     else cout<<"none"<<endl;
43     return 0;
44 }

```

4.4 Kruskal 重构树

Kruskal 重构树上任意两点 lca 权值代表两点间最大边权最小值

Listing 30: Kruskal 重构树

```

1 struct DSU{
2     vector<ll>f;
3     DSU(){
4         DSU(int n){
5             f.resize(n+1);
6             iota(f.begin(),f.end(),0);
7         }
8         int find(int x){
9             return x==f[x]?x:f[x]=find(f[x]);
10        }
11        bool merge(int x,int y){
12            int fx=find(x),fy=find(y);
13            if(fx==fy) return false;
14            f[fx]=fy;
15            return true;
16        }
17    };
18    struct Edge{
19        int x,y,ll w;
20        Edge(){
21            Edge(int x,int y,ll w):x(x),y(y),w(w){
22                friend bool operator<(Edge A,Edge B){
23                    return A.w<B.w;
24                }
25            };
26        inline void solve(){
27            int n,m;
28            cin>>n>>m;
29            DSU dsu(n<<1);
30            vector<vector<int>>G(n<<1|1);
31            vector<Edge>a;
32            vector<ll>val(n<<1|1); // 存新节点(边)权值
33            for(int i=1;i<=m;i++){
34                int x,y,ll w;
35                cin>>x>>y>>w;
36                a.emplace_back(x,y,w);

```

```

37     }
38     sort(a.begin(),a.end());
39     int tot=n;
40     for(auto &[x,y,w]:a){
41         int fx=dsu.find(x),fy=dsu.find(y);
42         if(fx!=fy){
43             ++tot;
44             dsu.merge(fx,tot);
45             dsu.merge(fy,tot);
46             G[tot].push_back(fx);
47             G[tot].push_back(fy);
48             val[tot]=w;
49         }
50     }
51     vector<int>dfn(n<<1|1);
52     vector<ll>d(n<<1|1);
53     vector<vector<int>>>st(n<<1|1,vector<int>(20));
54     int top=0;
55     auto dfs=[&](auto &&dfs,int u,int fa) ->void {
56         d[u]=d[fa]+1;
57         dfn[u]=++top;
58         st[dfn[u]][0]=fa;
59         for(int v:G[u]) dfs(dfs,v,u);
60     };
61     for(int i=1;i<=(n<<1);i++){
62         if(dsu.find(i)==i){
63             dfs(dfs,i,0);
64         }
65     }
66     for(int i=1;i<20;i++){
67         for(int j=1;j+(1<<i)-1<=(n<<1);j++){
68             int L=st[j][i-1];
69             int R=st[j+(1<<i-1)][i-1];
70             st[j][i]=(d[L]<d[R])?L:R;
71         }
72     }
73     auto lca=[&](int l,int r) ->int {
74         if(l==r) return l;
75         l=dfn[l],r=dfn[r];
76         if(l>r) swap(l,r);
77         ++l;
78         int len=_lg(r-l+1);
79         int L=st[l][len];
80         int R=st[r-(1<<len)+1][len];
81         return (d[L]<d[R])?L:R;
82     };
83     int q;cin>>q;
84     for(int i=1;i<=q;i++){
85         int x,y;cin>>x>>y;
86         if(dsu.find(x)!=dsu.find(y)) cout<<"impossible"<<
endl;
87         else cout<<val[lca(x,y)]<<endl;
88     }
89 }

```

Listing 31: 有向图欧拉回路

```

1 void solve(){
2     int n,m;
3     cin>>n>>m;

```

```

4     vector<vector<int>>>G(n+1);
5     vector<int>nxt(n+1);
6     vector<int>in(n+1),out(n+1);
7     for(int i=1;i<=m;i++){
8         int x,y;
9         cin>>x>>y;
10        G[x].emplace_back(y);
11        ++in[y],++out[x];
12    }
13    auto find=[&]() ->int {
14        int cnt1=0,cnt2=0;
15        for(int i=1;i<=n;i++){
16            if(in[i]-out[i]<-1||in[i]-out[i]>1) return
-1;
17            cnt1+=(in[i]-out[i]==-1);
18            cnt2+=(in[i]-out[i]==1);
19        }
20        if(cnt1+cnt2==0){
21            for(int i=1;i<=n;i++){
22                if(out[i]>0) return i;
23            }
24            //存在欧拉回路,需要保证图联通
25            else if(cnt1==1&&cnt2==1){
26                for(int i=1;i<=n;i++){
27                    if(out[i]-in[i]==1) return i;
28                }
29            } //存在欧拉路径
30            return -1;
31        }; //找到起点
32        int s=find();
33        if(s==-1){
34            cout<<"No"<<endl;
35            return;
36        }
37        vector<int>res;
38        vector<char>vis(m+1);
39        for(int i=1;i<=n;i++) sort(G[i].begin(),G[i].end());
40        auto dfs=[&](auto &&dfs,int u) ->void {
41            while(nxt[u]<G[u].size()){
42                int v=G[u][nxt[u]];
43                nxt[u]++; //当前弧优化
44                dfs(dfs,v);
45            }
46            res.push_back(u);
47        };
48        dfs(dfs,s);
49        if(res.size()!=m+1){
50            cout<<"No"<<endl;
51            return;
52        }
53        reverse(res.begin(),res.end());
54        for(int x:res) cout<<x<<' ';
55        cout<<endl;
56    }

```

Listing 32: 无向图欧拉回路

```

1 void solve(){
2     int m,n=500;cin>>m;
3     vector<vector<pair<int,int>>>G(n+1);
4     vector<int>in(n+1);
5     for(int i=1;i<=m;i++){
6         int x,y;cin>>x>>y;

```

```

7      G[x].emplace_back(y,i);
8      G[y].emplace_back(x,i);
9      ++in[x],++in[y];
10 }
11 auto find=[&]() ->int {
12     int cnt=0;
13     for(int i=1;i<=n;i++){
14         if(in[i]&1) ++cnt;
15     }
16     if(cnt!=2&&cnt!=0) return -1;
17     else if(cnt==0) {
18         for(int i=1;i<=n;i++){
19             if(in[i]) return i;
20         }
21     }
22     else if(cnt==2){
23         for(int i=1;i<=n;i++){
24             if(in[i]&1) return i;
25         }
26     }
27     return -1;
28 };
29 int s=find();
30 vector<int>nxt(n+1),vis(m+1),res;
31 for(int i=1;i<=n;i++) sort(G[i].begin(),G[i].end());
32 auto dfs=[&](auto &&dfs,int u) ->void {
33     while(nxt[u]<G[u].size()){
34         int v=G[u][nxt[u]].first;
35         int id=G[u][nxt[u]].second;
36         ++nxt[u];
37         if(vis[id]) continue;
38         vis[id]=1;
39         dfs(dfs,v);
40     }
41     res.push_back(u);
42 };
43 dfs(dfs,s);
44 reverse(res.begin(),res.end());
45 for(int x:res) cout<<x<<endl;
46 }

```

4.6 二分图

4.6.1 二分图染色

Listing 33: 二分图染色

```

1 void solve(){
2     int n,m;
3     cin>>n>>m;
4     vector<vector<int>>>G(n+1);
5     for(int i=1;i<=m;i++){
6         int x,y;cin>>x>>y;
7         G[x].emplace_back(y);
8         G[y].emplace_back(x);
9     }
10    int A=0,B=0;
11    vector<int>col(n+1);
12    auto dfs=[&](auto &&dfs,int u) ->bool {
13        bool f=true;
14        for(int v:G[u]){
15            if(!col[v]) {
16                col[v]=3-col[u];

```

```

17                if(col[v]==1) ++A;
18                else ++B;
19                f&=dfs(dfs,v);
20            }
21            else if(col[v]!=3-col[u]) f=false;//当前联通分量无法染色
22        }
23        return f;
24    };
25    ll ans=0;
26    for(int i=1;i<=n;i++){
27        if(!col[i]){
28            A=B=0;
29            col[i]=1;
30            ++A;
31            if(dfs(dfs,i)) ans+=max(A,B);
32        }
33    }
34    cout<<ans<<endl;
35 }

```

4.6.2 二分图最大权匹配

Listing 34: 二分图最大匹配

```

1 const int N=505;
2 vector<int>G[N];
3 int match[N];
4 bool vis[N];
5 bool dfs(int u){
6     for(auto &v:G[u]){
7         if(vis[v]) continue;
8         vis[v]=1;
9         if(!match[v]||dfs(match[v])){
10             match[v]=u;
11             return 1;
12         }
13     }
14     return 0;
15 }
16 int main(){
17     cin.tie(0)->ios::sync_with_stdio(false);
18     int n,m,e;cin>>n>>m>>e;
19     for(int i=1;i<=e;i++){
20         int u,v;
21         cin>>u>>v;
22         G[u].push_back(v);
23     }
24     ll ans=0;
25     for(int i=1;i<=n;i++) {
26         fill(vis+1,vis+1+n,false);
27         ans+=dfs(i);
28     }
29     cout<<ans<<endl;
30     return 0;
31 }

```

4.7 差分约束

给定 n 个变量 $x_1, x_2, \dots, x_n$ 与 m 个差分约束

$$x_c - x_{c'} \leq y。$$

求任意一组满足所有约束的可行解。无解输出 NO。

Listing 35: spfa 差分约束

```
1  const int N=2e5+5;
2  struct node{
3      int u,ll w;
4  };
5  vector<node>G[N];
6  int cnt[N];
7  bool inq[N];
8  ll dis[N];
9  int n,m;
10 bool spfa(){
11     fill(dis+1,dis+1+n,INT_MAX);
12     queue<int>Q;
13     Q.push(0);
14     inq[0]=0;
15     cnt[0]=1;
16     while(!Q.empty()){
17         int u=Q.front();
18         Q.pop();
19         inq[u]=false;
20         for(auto &[v,w]:G[u]){
21             if(dis[v]>dis[u]+w){
22                 dis[v]=dis[u]+w;
23                 if(!inq[v]){
24                     cnt[v]++;
25                     if(cnt[v]>n) return false;
26                     Q.push(v);
27                     inq[v]=1;
28                 }
29             }
30         }
31     }
32     return true;
33 }
34 void solve(){
35     cin>>n>>m;
36     for(int i=1;i<=m;i++){
37         int x,y,ll w;cin>>x>>y>>w;
38         G[y].push_back({x,w});
39     }
40     for(int i=1;i<=n;i++){
41         G[0].push_back({i,0});
42     }
43     if(spfa()){
44         for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
45         cout<<endl;
46     }
47     else cout<<"NO"<<endl;
48 }
```

4.8 缩点

4.8.1 SCC(强连通分量)

Listing 36: SCC

```
1  const int N=1e4+5;
2  vector<int>G[N];
3  int w[N],new_w[N];
```

```
4  vector<int>new_G[N];
5  struct SCC{
6      vector<vector<int>>ans;
7      vector<int>dfn,low,mp;
8      vector<char>vis;
9      stack<int>st;
10     int cnt,n;
11     SCC(int n):n(n),cnt(0){
12         vis.resize(n+1);
13         dfn.resize(n+1);
14         low.resize(n+1);
15         mp.resize(n+1);
16     }
17     void tarjan(int u){
18         low[u]=dfn[u]=(++cnt);
19         st.push(u);
20         vis[u]=1;
21         for(int v:G[u]){
22             if(!dfn[v]){
23                 tarjan(v);
24                 low[u]=min(low[u],low[v]);
25             }
26             else if(vis[v]){
27                 low[u]=min(low[u],dfn[v]);
28             }
29         }
30         if(dfn[u]==low[u]){
31             int y;
32             vector<int>tmp;
33             do{
34                 y=st.top();
35                 st.pop();
36                 vis[y]=0;
37                 tmp.push_back(y);
38                 mp[y]=ans.size();
39             }while(u!=y);
40             if(!tmp.empty()) ans.push_back(tmp);
41         }
42     }
43 };
44 void solve(){
45     int n,m;
46     cin>>n>>m;
47     for(int i=1;i<=n;i++) cin>>w[i];
48     for(int i=1;i<=m;i++){
49         int x,y;cin>>x>>y;
50         G[x].push_back(y);
51     }
52     SCC scc(n+1);
53     for(int i=1;i<=n;i++){
54         if(!scc.dfn[i]) scc.tarjan(i);
55     }
56     for(int i=0;i<scc.ans.size();i++){
57         for(int v:scc.ans[i]){
58             new_w[i]+=w[v];
59         }
60     }
61     vector<int>in(scc.ans.size(),dp(scc.ans.size()));
62     for(int i=1;i<=n;i++){
63         for(int v:G[i]){
64             if(scc.mp[v]!=scc.mp[i]){
65                 new_G[scc.mp[i]].push_back(scc.mp[v]);
```

```

66         ++in[scc.mp[v]];
67     }
68 }
69 }
70 queue<int>Q;
71 for(int i=0;i<scc.ans.size();i++){
72     if(!in[i]) {
73         Q.push(i);
74         dp[i]=new_w[i];
75     }
76 }
77 while(!Q.empty()){
78     int u=Q.front();
79     Q.pop();
80     for(int v:new_G[u]){
81         if((--in[v])==0){
82             dp[v]=max(dp[v],dp[u]+new_w[v]);
83             Q.push(v);
84         }
85     }
86 }
87 cout<<*max_element(dp.begin(),dp.end())<<endl;
88 }

```

4.8.2 DECC(边双联通分量)

Listing 37: DECC

```

1  const int N=2e5+5;
2  vector<pair<int,int>>G[N];
3  struct DECC{
4      vector< vector<pair<int,int> >>G;
5      vector< vector<int> >B; // 存储所有边双连通分量
6      vector<int>dfn,low; // 时间戳
7      stack<int>st;
8      vector<char>vis;
9      vector<int>mp; // 每个点属于哪个分量
10     int cnt,n,id;
11     DECC(int n):n(n),cnt(0),id(0){
12         G.resize(n+1);
13         low.resize(n+1);
14         dfn.resize(n+1);
15         mp.resize(n+1);
16     }
17     void add(int x,int y){
18         ++id;
19         G[x].push_back({y,id});
20         G[y].push_back({x,id});
21     }
22     void tarjan(int u,int fa){
23         dfn[u]=low[u]=(++cnt);
24         st.push(u);
25         for(auto &[v,id]:G[u]){
26             if(!dfn[v]){
27                 tarjan(v,id);
28                 low[u]=min(low[u],low[v]);
29             }
30             else if(id!=fa&&dfn[v]<dfn[u]){
31                 low[u]=min(low[u],dfn[v]);
32             }
33         }
34         if(dfn[u]==low[u]){
35             vector<int>tmp;

```

```

36         int y;
37         do{
38             y=st.top();
39             mp[y]=B.size();
40             st.pop();
41             tmp.push_back(y);
42         }while(y!=u);
43         if(!tmp.empty()) B.push_back(tmp);
44     }
45 }
46 };
47 void solve(){
48     int n,m;
49     cin>>n>>m;
50     DECC bcc(n+1);
51     for(int i=1;i<=m;i++){
52         int x,y;cin>>x>>y;
53         bcc.add(x,y);
54     }
55     for(int i=1;i<=n;i++){
56         if(!bcc.dfn[i]) bcc.tarjan(i,0);
57     }
58     cout<<bcc.B.size()<<endl;
59     for(auto &p:bcc.B){
60         cout<<p.size()<<' ';
61         for(int &x:p) cout<<x<<' ';
62         cout<<endl;
63     }
64 }

```

5 字符串

5.1 字符串哈希

Listing 38: 字符串哈希

```
1 using ull = unsigned long long;
2 ull base = 131;
3 ull mod1 = 212370440130137957, mod2 = 1e9 + 7;
4 ull get_hash1(const string &s) {
5     int len = s.size();
6     ull ans = 0;
7     for (int i = 0; i < len; i++) ans = (ans * base + (ull)
8         s[i]) % mod1;
9     return ans;
10 }
11 ull get_hash2(const string &s) {
12     int len = s.size();
13     ull ans = 0;
14     for (int i = 0; i < len; i++) ans = (ans * base + (ull)
15         s[i]) % mod2;
16     return ans;
17 }
18 bool cmp(const string &s, const string &t) {
19     bool f1 = get_hash1(s) != get_hash1(t);
20     bool f2 = get_hash2(s) != get_hash2(t);
21     return f1 || f2;
22 }
```

5.2 KMP

Listing 39: KMP1

```
1 vector<int>Kmp(string s){
2     vector<int>pi(s.size()+1);
3     for(int i=1;i<s.size();i++){
4         int j=pi[i-1];
5         while(j&& s[j]!=s[i]) j=pi[j-1];
6         if(s[i]==s[j]) j++;
7         pi[i]=j;
8     }
9     return pi;
10 }
11 void solve(){
12     string s,t;
13     cin>>t>>s;
14     string res=s+"#"+t;
15     vector<int>pi=Kmp(res);
16     vector<int>ans=Kmp(s);
17     for(int i=s.size();i<pi.size();i++){
18         if(pi[i]==s.size()) cout<<i-s.size()*2+1<<endl;
19     }
20     for(int i=0;i<s.size();i++) cout<<ans[i]<<' ';
21     cout<<endl;
22 }
```

Listing 40: KMP2

```
1 vector<int>border(const string &s){
2     int n=s.size();
3     vector<int>pi(n);
4     for(int i=1;i<n;i++){
5         int j=pi[i-1];
```

```
6         while(j>0&&s[j]!=s[i]) j=pi[j-1];
7         if(s[i]==s[j]) j++;
8         pi[i]=j;
9     }
10    return pi;
11 }
12 void kmp(const string &s,const string &t){
13     vector<int>pi=border(t);
14     int n=s.size(),m=t.size();
15     for(int i=0,j=0;i<n;i++){
16         while(j>0&&s[i]!=t[j]) j=pi[j-1];
17         if(s[i]==t[j]) j++;
18         if(j==m){
19             j=pi[j-1];
20             cout<<i-m+2<<endl;
21         }
22     }
23     for(int x:pi) cout<<x<<' ';
24     cout<<endl;
25 }
26 int main(){
27     cin.tie(0)->ios::sync_with_stdio(false);
28     string s,t;
29     cin>>s>>t;
30     kmp(s,t);
31     return 0;
32 }
```

5.3 Manacher(马拉车)

Listing 41: 马拉车求最长回文子串

```
1 int Manacher(const string& st) {
2     string s = "~$";
3     for (char c : st) {
4         s += c;
5         s += '$';
6     }
7     s += '&';
8     int len = s.size(), mid = 1, mx = 1, ans = -1;
9     vector<int> p(len);
10    for (int i = 1; i < len; ++i) {
11        if (i < mx)
12            p[i] = min(mx - i, p[2 * mid - i]);
13        else
14            p[i] = 1;
15        while (s[i - p[i]] == s[i + p[i]])
16            ++p[i];
17        if (mx < i + p[i]) {
18            mid = i;
19            mx = i + p[i];
20        }
21        ans = max(ans, p[i] - 1);
22    }
23    return ans;
24 }
25 int main() {
26     string st;
27     cin >> st;
28     cout << Manacher(st) << endl;
29 }
```

6 动态规划

6.1 01 背包

Listing 42: 01 背包

```
1 void solve(){
2     int w,n;cin>>w>>n;
3     vector<int>a(n+1),b(n+1);
4     vector<int>f(w+1);
5     for(int i=1;i<=n;i++) cin>>a[i]>>b[i];
6     for(int i=1;i<=n;i++) {
7         for(int j=w;j>=a[i];j--){
8             f[j]=max(f[j-a[i]]+b[i],f[j]);
9         }
10    }
11    cout<<f[w]<<endl;
12 }
```

Listing 43: bitset 优化可行 01 背包

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 void solve(){
5     int w, n; cin >> w >> n;
6     bitset<10001> dp;
7     dp[0] = 1;
8     for(int i = 1; i <= n; i++) {
9         int a; cin >> a;
10        dp |= dp << a;
11    }
12    for(int j = w; j >= 0; j--) {
13        if(dp[j]) {
14            cout << j << endl;
15            return;
16        }
17    }
18 }
```

Listing 44: 双向搜索优化纯搜索

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 vector<pair<ll, int>> gen(vector<ll> a) {
5     vector<pair<ll, int>> res;
6     int k = a.size();
7     for (int mask=0; mask<(1<<k); mask++) {
8         ll s=0;
9         for (int i=0; i<k; i++) if (mask&(1<<i)) s += a[i];
10    };
11    res.emplace_back(s, mask);
12    sort(res.begin(), res.end());
13    return res;
14 }
15
16 int main() {
17     int n; ll c;
18     scanf("%d%lld", &n, &c);
19     vector<ll> w(n);
20     for (int i=0; i<n; i++) scanf("%lld", &w[i]);
```

```
21     int mid = n/2;
22     vector<ll> L(w.begin(), w.begin()+mid), R(w.begin()+
23     mid, w.end());
24     auto ls = gen(L), rs = gen(R);
25     ll maxw=0; int lm=0, rm=0;
26     for (auto [sb, m]: rs) {
27         if (sb > c) continue;
28         ll rem = c - sb;
29         int pos = upper_bound(ls.begin(), ls.end(),
30         make_pair(rem, (1<<mid)-1)) - ls.begin() - 1;
31         if (pos >= 0 && ls[pos].first + sb > maxw) {
32             maxw = ls[pos].first + sb;
33             lm = ls[pos].second;
34             rm = m;
35         }
36     }
37     vector<int> ans(n);
38     for (int i=0; i<mid; i++) ans[i] = (lm >> i) & 1;
39     for (int i=0; i<n-mid; i++) ans[mid+i] = (rm >> i) &
40     1;
41     printf("%lld\n", maxw); //最优代价
42     for (int i=0; i<n; i++) printf("%d ", ans[i]); //最优
43     方案输出
44     return 0;
45 }
```

6.2 完全背包

Listing 45: 朴素完全背包

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 int main(){
6     cin.tie(0)->ios::sync_with_stdio(false);
7     int n,w;cin>>w>>n;
8     vector<ll>a(n+1),b(n+1),f(w+1);
9     for(int i=1;i<=n;i++) cin>>a[i]>>b[i];
10    for(int i=1;i<=n;i++){
11        for(int j=a[i];j<=w;j++){
12            f[j]=max(f[j-a[i]]+b[i],f[j]);
13        }
14    }
15    cout<<f[w]<<endl;
16    return 0;
17 }
```

6.3 多重背包

Listing 46: 二进制优化多重背包

```
1 const ll INF=1e15;
2 void solve(){
3     int n,w;
4     cin>>n>>w;
5     vector<ll>dp(w+1,INF);
6     dp[0]=0;
7     for(int i=1;i<=n;i++){
8         int l,c;cin>>l>>c;
9         for(int k=1;c>0;k*=2){
10             ll cur=min(c,k);
```



```

11         for(int j=w;j>=1LL*cur*1;j--){
12             dp[j]=min(dp[j],dp[j-1LL*cur*1]+cur);
13         }
14         c-=cur;
15     }
16 }
17 if(dp[w]>=INF) cout<<-1<<endl;
18 else cout<<dp[w]<<endl;
19 }
20 int main(){
21     cin.tie(0)->ios::sync_with_stdio(false);
22     solve();
23     return 0;
24 }

```

6.4 树上背包

25 上海市赛 I

题目大意

给定一棵以 1 为根的有根树，每个节点上的人要么是“诚实者”（恒说真话），要么是“说谎者”（恒说假话）。每个节点 i 的人声称：“以我为根的子树中，恰好有 a_i 个诚实者。”求所有满足以下逻辑自洽条件的身份分配方案数：

- 若节点 i 是诚实者，则其子树中诚实者的实际数量必须等于 a_i ；
- 若节点 i 是说谎者，则其子树中诚实者的实际数量必须不等于 a_i 。
- 第一行：一个整数 T ($1 \leq T \leq 5000$)，表示数据组数。
- 每组数据第一行：一个正整数 n ($1 \leq n \leq 5000$, $\sum n \leq 5000$)，表示节点数量。
- 每组数据第二行： n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq n$)，表示每个节点的陈述。
- 接下来 $n - 1$ 行：每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示树的一条边，保证输入构成一棵以 1 为根的有根树。

输出格式

对每组数据输出一行一个整数，表示满足条件的真假分配情况数量，答案对 998244353 取模。

简要分析

采用树形背包 DP 求解，通过后序遍历合并子树状态，根据节点身份（诚实/说谎）约束转移，统计合法方案数。

状态定义

- $f[u][j]$: 仅用节点 u 的子树，产生 j 个诚实者的合法方案数（不包含 u 自身）；
- $dp[u][j]$: 用节点 u 的子树 + u 自身，产生 j 个诚实者的合法方案数；
- $siz[u]$: 以 u 为根的子树大小。

状态转移

1. 初始化: $f[u][0] = 1$, $siz[u] = 1$; 2. 子树合并: 遍历子节点 v ，用 ** 树形背包 ** 倒序枚举合并状态：

$$nxt[j+k] += f[u][j] \times dp[v][k] \pmod{mod}$$

用 nxt 更新 $f[u]$ ，并更新子树大小；3. 身份转移：
- 说谎者: $dp[u][j] = f[u][j]$ ($j \neq a[u]$)；
- 诚实者: $dp[u][a[u]] = f[u][a[u] - 1]$ ($a[u] \geq 1$)。

复杂度分析

代码虽为三重循环，本质是树形背包均摊复杂度：任意两个节点仅在其最近公共祖先处被枚举一次，总操作数等于子树大小乘积和，** 严格为 $O(n^2)$ **；结合题目约束 $\sum n \leq 5000$ ，总计算量不超过 2.5×10^7 ，可轻松通过。

答案统计

最终答案为根节点 1 的所有合法状态之和：

$$\sum_{i=0}^{siz[1]} dp[1][i] \pmod{998244353}.$$

Listing 47: 树上背包

```

1  const ll mod=998244353;
2  void solve(){
3      int n;cin>>n;
4      vector<int>a(n+1);
5      vector< vector<int> >G(n+1);
6      for(int i=1;i<=n;i++) cin>>a[i];
7      for(int i=1;i<n;i++){
8          int x,y;cin>>x>>y;
9          G[x].emplace_back(y);
10         G[y].emplace_back(x);
11     }

```

```

12     vector<vector<ll>>dp(n+1,vector<ll>(n+1)); //使用子树
    和当前点产生j个诚实者的合法情况
13     vector<vector<ll>>f(n+1,vector<ll>(n+1)); //仅使用子树
    产生j个诚实者的合法情况
14     vector<int>siz(n+1);
15     auto dfs=[&](auto &&dfs,int u,int fa) ->void {
16         f[u][0]=siz[u]=1;
17         for(int v:G[u]){
18             if(v==fa) continue;
19             dfs(dfs,v,u);
20             vector<ll>nxt(siz[u]+siz[v]+1);
21             for(int j=siz[u];j>=0;j--){
22                 for(int k=siz[v];k>=0;k--){
23                     (nxt[j+k]+=(f[u][j]*dp[v][k]%mod))%=
mod;
24                 }
25             }
26             f[u]=move(nxt);
27             siz[u]+=siz[v];
28         }
29         for(int j=siz[u];j>=0;j--) dp[u][j]=f[u][j];
30         dp[u][a[u]]=0;
31         if(a[u]>=1&& a[u]<=siz[u]) {
32             dp[u][a[u]]=f[u][a[u]-1];
33         }
34     };
35     dfs(dfs,1,0);
36     ll ans=0;
37     for(int i=0;i<=siz[1];i++) (ans+=dp[1][i])%=mod;
38     cout<<ans<<endl;
39 }

```

6.5 换根 dp

Listing 48: 换根 dp

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 const int N=1e6+5;
6 vector<ll>G[N];
7 vector<ll>ans,siz;
8 void add(int x,int y){
9     G[x].emplace_back(y);
10    G[y].emplace_back(x);
11 }
12 void dfs(int u,int f){
13     siz[u]=1;
14     for(auto &v:G[u]){
15         if(v==f) continue;
16         dfs(v,u);
17         siz[u]+=siz[v];
18     }
19 }
20 void __dfs(int u,int f){
21     for(auto &v:G[u]){
22         if(v==f) continue;
23         ans[v]=(ans[u]+siz[1]-siz[v]*2);
24         __dfs(v,u);
25     }
26 }

```

```

27 int main(){
28     cin.tie(0)->ios::sync_with_stdio(false);
29     int n;cin>>n;
30     ans.resize(n+1);
31     siz.resize(n+1);
32     for(int i=1;i<n;i++){
33         int x,y;cin>>x>>y;
34         add(x,y);
35     }
36     dfs(1,0);
37     ans[1]=siz[1];
38     __dfs(1,0);
39     cout<<max_element(ans.begin()+1,ans.begin()+1+n)-ans.
begin()<<endl;
40     return 0;
41 }

```

6.6 区间 dp

Listing 49: 区间 dp 典

```

1 //P1995 石子合并
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 const int N=105;
7 const ll INF=1e12;
8 ll a[N<<1],pre[N<<1];
9 ll Mx[N<<1][N<<1],Mn[N<<1][N<<1];
10 ll mx(int l,int r){
11     if(Mx[l][r]!=-1) return Mx[l][r];
12     if(l==r) return a[l];
13     ll ans=0;
14     for(int k=l;k+1<=r;k++){
15         ans=max(ans,mx(l,k)+mx(k+1,r));
16     }
17     return Mx[l][r]=ans+(pre[r]-pre[l-1]);
18 }
19 ll mn(int l,int r){
20     if(Mn[l][r]!=-1) return Mn[l][r];
21     if(l==r) return a[l];
22     ll ans=INF;
23     for(int k=l;k+1<=r;k++){
24         ans=min(ans,mn(l,k)+mn(k+1,r));
25     }
26     return Mn[l][r]=ans+(pre[r]-pre[l-1]);
27 }
28 void solve(){
29     int n;cin>>n;
30     for(int i=1;i<=n;i++) cin>>a[i],a[i+n]=a[i];
31     for(int i=1;i<=n*2;i++) pre[i]=pre[i-1]+a[i];
32     memset(Mx,-1,sizeof Mx);
33     memset(Mn,-1,sizeof Mn);
34     ll ans1=0,ans2=INF;
35     for(int i=1;i<=n;i++){
36         int l=i,r=n+i-1;
37         ans1=max(ans1,mx(l,r)-(pre[r]-pre[l-1]));
38         ans2=min(ans2,mn(l,r)-(pre[r]-pre[l-1]));
39     }
40     cout<<ans2<<endl<<ans1<<endl;
41 }

```

```

42 int main(){
43     cin.tie(0)->ios::sync_with_stdio(false);
44     solve();
45     return 0;
46 }

```

6.7 SOS dp

例

给定长度为 N 的整数序列 $A = (A_1, A_2, \dots, A_N)$, 求满足 $1 \leq i < j \leq N$ 且列加法计算 $A_i + A_j$ 时不发生进位的整数对 (i, j) 的个数。

限制条件

- $1 \leq N \leq 100$
- $1 \leq W \leq 50000$
- $1 \leq L_i \leq W$
- $1 \leq C_i \leq 10000$
- 所有输入均为整数

Listing 50: SOSdp 解决数位不发生进位问题

```

1 //ARC136D
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 const int N=1e6+1;
7 int Flip(int x){
8     int p=1,res=0;
9     for(int i=0;i<6;i++){
10         res+=(9-x%10)*p;
11         x/=10;
12         p*=10;
13     }
14     return res;
15 }//这个数的反集,如123->876
16 bool judge(int x){
17     while(x){
18         if(x%10>=5) return false;
19         x/=10;
20     }
21     return true;
22 }//判定这个数与自身相加是否发生进位情况
23 int main(){
24     cin.tie(0)->ios::sync_with_stdio(false);
25     int n;cin>>n;
26     vector<ll>dp(N);
27     vector<int>a(n+1);
28     for(int i=1,x;i<=n;i++){
29         cin>>a[i];
30         dp[a[i]]++;
31     }
32     int p=1;

```

```

33     for(int i=1;i<=6;i++){//本质是对每个十进制数位进行一次
        强缀和,又称高维前缀和(SOS dp)
34         for(int j=0;j<N;j++){
35             if((j/p)%10) dp[j]+=dp[j-p];
36         }
37         p*=10;
38     }
39     ll ans=0;
40     for(int i=1;i<=n;i++){
41         int x=Flip(a[i]);
42         ans+=dp[x];
43         if(judge(a[i])) ans--;
44     }
45     cout<<ans/2<<endl;
46     return 0;
47 }

```

6.8 数位 dp

Listing 51: 统计 a-b 中各个数位出现个数

```

1 ll dp[18][18][2][2];
2 int digit[18],target;
3 ll dfs(int pos,int cnt,bool limit,bool lead){
4     if(pos==-1) return cnt;
5     if(dp[pos][cnt][limit][lead]!=-1) return dp[pos][cnt]
6     ] [limit][lead];
7     int up=(limit)?digit[pos]:9;
8     ll res=0;
9     for(int i=0;i<=up;i++){
10         int ncnt=cnt;
11         if(i==target){
12             if(!(lead&& i==0)){
13                 ncnt++;
14             }
15             bool nlead=lead&&(i==0);
16             bool nlimit=limit&&(i==up);
17             res+=dfs(pos-1,ncnt,nlimit,nlead);
18         }
19         return dp[pos][cnt][limit][lead]=res;
20     }
21 ll solve(ll x){
22     if(x<0) return 0;
23     int len=0;
24     while(x>0){
25         digit[len++]=x%10;
26         x/=10;
27     }
28     memset(dp,-1,sizeof(dp));
29     return dfs(len-1,0,true,true);
30 }
31 int main(){
32     cin.tie(0)->ios::sync_with_stdio(false);
33     ll a,b;cin>>a>>b;
34     for(int i=0;i<10;i++){
35         target=i;
36         cout<<solve(b)-solve(a-1)<<" ";
37     }
38     cout<<endl;
39     return 0;
40 }

```

7 数学

7.1 筛法求因数个数

Listing 52: 因数筛

```
1  const int N = 1000010;
2  int p[N];    // p[i]: i 的最小质因子 (spf)
3  int pr[N];   // pr[]: 质数表
4  int cnt;
5  int d[N];    // d[i]: i 的正因数个数
6  int a[N];    // a[i]: i 的最小质因子的指数
7  void init(int n) {
8      d[1] = 1; a[1] = 0; p[1] = 1;
9      for(int i = 2; i <= n; i++) {
10         if(!p[i]) {                // i 是质数
11             p[i] = pr[++cnt] = i;
12             d[i] = 2;
13             a[i] = 1;
14         }
15         for(int j = 1; j <= cnt && 1LL * i * pr[j] <= n;
16             j++) {
17             int m = i * pr[j];
18             p[m] = pr[j];           // 记录 m 的最小质
19             因子
20             if(p[i] == pr[j]) {     // 还是同一个质因子
21                 d[m] = d[i] / (a[i] + 1) * (a[i] + 2);
22                 a[m] = a[i] + 1;
23                 break;
24             }
25             else {                 // 新增一个不同质因
26                 子
27                 d[m] = d[i] * 2;    // 等价于 d[i] * d
28                 [pr[j]]
29                 a[m] = 1;
30             }
31         }
32     }
33 }
```

7.2 筛法求欧拉函数

Listing 53: 欧拉筛

```
1  const int N=1e6+5;
2  int p[N],vis[N],cnt;
3  int phi[N];
4  void init(){
5      phi[1]=1;
6      for(int i=2;i<=n;i++){
7          if(!vis[i]){
8              p[cnt++]=i;
9              phi[i]=i-1;
10         }
11         for(int j=0;i*p[j]<=n;j++){
12             int m=i*p[j];
13             vis[m]=1;
14             if(i%p[j]==0){
15                 phi[m]=p[j]*phi[i];
16                 break;
17             }
18             else phi[m]=(p[j]-1)*phi[i];
19         }
20     }
21 }
```

```
19     }
20 }
21 }
```

7.3 筛法求莫比乌斯函数

Listing 54: 莫比乌斯函数筛

```
1  const int N=1e6+5;
2  int p[N],vis[N],cnt;
3  int mu[N];
4  void init(){
5      mu[1]=1;
6      for(int i=2;i<=n;i++){
7          if(!vis[i]){
8              p[cnt++]=i;
9              mu[i]=-1;
10         }
11         for(int j=0;i*p[j]<=n;j++){
12             int m=i*p[j];
13             vis[m]=1;
14             if(i%p[j]==0){
15                 mu[m]=0;
16                 break;
17             }
18             else{
19                 mu[m]=-mu[i];
20             }
21         }
22     }
23 }
```

7.4 矩阵快速幂

Listing 55: 矩阵快速幂 basic

```

1 struct Mat{
2     vector<vector<ll>>a;
3     int n;
4     Mat(int n):n(n){
5         a=vector<vector<ll>>(n+1,vector<ll>(n+1));
6     };
7     Mat(){};
8     friend Mat operator+(Mat &A,Mat &B){
9         Mat C(A.n);
10        for(int i=0;i<=A.n;i++){
11            for(int j=0;j<=A.n;j++){
12                C.a[i][j]=(A.a[i][j]+B.a[i][j])%mod;
13            }
14        }
15        return C;
16    }
17    friend Mat operator+=(Mat &A,Mat &B){
18        A=A+B;
19        return A;
20    }
21    friend Mat operator*(Mat &A,Mat &B){
22        Mat C(A.n);
23        for(int k=0;k<=A.n;k++){
24            for(int i=0;i<=A.n;i++){
25                if(A.a[i][k]==0) continue;
26                for(int j=0;j<=A.n;j++){
27                    C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k
28                ] [j])%mod;
29            }
30        }
31        return C;
32    }
33    friend Mat operator*=(Mat &A,Mat &B){
34        A=A*B;
35        return A;
36    }
37    void q_pow(Mat &A,ll B){
38        Mat S(A.n); //单位矩阵
39        for(int i=0;i<=A.n;i++) S.a[i][i]=1;
40        while(B>0){
41            if(B&1){
42                S*=A;
43            }
44            A*=A;
45            B>>=1;
46        }
47        A=move(S);
48    }
49 }M;

```

Listing 56: 矩阵快速幂 max-plus

```

1 const ll INF=1e18;
2 struct Mat{
3     vector<vector<ll>>a;
4     int n;
5     Mat(int n):n(n){
6         a=vector<vector<ll>>(n+1,vector<ll>(n+1,-INF));

```

```

7     };
8     Mat(){};
9     friend Mat operator*(const Mat &A,const Mat &B){
10        Mat C(A.n);
11        for(int k=0;k<=A.n;k++){
12            for(int i=0;i<=A.n;i++){
13                for(int j=0;j<=A.n;j++){
14                    C.a[i][j]=max(C.a[i][j],A.a[i][k]+B.a
15                [k][j]);
16            }
17        }
18        return C;
19    }
20    friend Mat operator*=(Mat &A,Mat &B){
21        A=A*B;
22        return A;
23    }
24    void q_pow(Mat &A,ll B){
25        Mat S(A.n); //单位矩阵
26        for(int i=0;i<=A.n;i++) S.a[i][i]=0;
27        while(B>0){
28            if(B&1) S*=A;
29            A*=A;
30            B>>=1;
31        }
32        A=move(S);
33    }
34 }M;

```

7.5 第二类斯特林数

第二类斯特林数（斯特林子集数） $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ ，也可记做 $S(n, k)$ ，表示将 n 个两两不同的元素划分为 k 个互不区分的非空子集的方案数。

Listing 57: Stirling 数

```

1 typedef unsigned long long ll;
2 const int N=10010;
3 ll dp[N][N/10],p;
4 void init(){
5     dp[1][1]=1;
6     for(int i=2;i<=10000;i++){
7         for(int j=1;j<=1000;j++){
8             dp[i][j]=(j%p)*dp[i-1][j]%p+
9             dp[i-1][j-1]%p;
10        }
11    }
12    int main(){
13        cin.tie(0)->ios::sync_with_stdio(false
14        );
15        int n,m;

```

```

15     cin>>n>>m>>p;
16     init();
17     cout<<dp[n][m]<<endl;
18     return 0;
19 }

```

7.6 卡特兰数

Catalan 数有如下常见的表达式:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{n!(n+1)!}, \quad n \geq 0. \quad (2)$$

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}, \quad n \geq 0. \quad (3)$$

$$C_n = \frac{4n-2}{n+1} C_{n-1}, \quad n > 0, \quad C_0 = 1. \quad (4)$$

数列的前几项为: 1,1,2,5,14,42,132,429,1430,...

7.7 排列组合类-含 Lucas

```

1  const ll mod=998244353;
2  vector<ll>fac,invfac;
3  ll q_pow(ll x,ll y){
4      ll s=1;
5      x%=mod;
6      while(y>0){
7          if(y&1){
8              s=s*x%mod;
9          }
10         x=x*x%mod;
11         y>>=1;
12     }
13     return s;
14 }
15 void init(int n){
16     fac.resize(n+1);
17     invfac.resize(n+1);
18     fac[0]=invfac[0]=1;
19     for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%mod;
20     invfac[n]=q_pow(fac[n],mod-2);
21     for(int i=n-1;i>=1;i--) invfac[i]=invfac[i+1]*(i+1)%
    mod;
22 }
23 ll C(ll n,ll m){
24     if(n<m) return 0;
25     if(n<mod&& m<mod) return fac[n]*invfac[m]%mod*invfac[n-
    m]%mod;
26     return C(n/mod,m/mod)*C(n%mod,m%mod)%mod;
27 }

```

7.8 高斯消元

Listing 58: 高斯消元

```

1  const int N=110;
2  const double eps=1e-8;
3  int n;
4  double a[N][N];
5  ll gauss(){
6      ll c,r;
7      for(c=0,r=0;c<n;c++){
8          ll t=r;
9          for(int i=r;i<n;i++){

```

```

10         if(abs(a[i][c])>abs(a[t][c])) t=i;
11     }
12     if(abs(a[t][c])<eps) continue;
13     for(int j=c;j<n+1;j++) swap(a[t][j],a[r][j]);//将
绝对值最大的换到最顶端
14     for(int j=n;j>=c;j--) a[r][j]/=a[t][c];//将当前行
首位变成1
15     for(int i=r+1;i<n;i++){
16         if(abs(a[i][c])>eps){
17             for(int j=n;j>=c;j--) a[i][j]-=a[r][j]*a[
i][c];
18         }
19     }
20     r++;
21 }
22 if(r<n){
23     for(int i=r;i<n;i++){
24         if(abs(a[i][n])>eps) return
2;
25     }
26     return 1;
27 }
28 for(int i=n-1;i>=0;i--){
29     for(int j=i+1;j<n;j++){
30         a[i][n]-=a[i][j]*a[j][n];
31     }
32     return 0;
33 }
34 void solve(){
35     cin>>n;
36     for(int i=0;i<n;i++){
37         for(int j=0;j<n+1;j++){
38             cin>>a[i][j];
39         }
40     }
41     ll t=gauss();
42     cout<<fixed<<setprecision(2);
43     if(t==0) {
44         for(int i=0;i<n;i++){
45             if(abs(a[i][n])<eps) a[i][n]=abs(a[i][n]);
46             cout<<a[i][n]<<endl;
47         }
48     }
49     else cout<<"No Solution"<<endl;
50 }

```

7.9 Pollard-Rho

Listing 59: Pollard-Rho

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using i128 = __int128;
5 // ===== 快速乘（取模） =====
6 ll mul_mod(ll a, ll b, ll mod) {
7     return (ll)((i128)a * b % mod);
8 }
9 // ===== 快速幂取模 =====
10 ll q_pow(ll a, ll d, ll mod) {
11     ll res = 1;
12     a %= mod;
13     while (d) {
14         if (d & 1) res = mul_mod(res, a, mod);
15         a = mul_mod(a, a, mod);

```

```

16         d >>= 1;
17     }
18     return res;
19 }
20 // ===== Miller-Rabin 确定性判素数 =====
21 bool is_prime(ll n) {
22     if (n < 2) return false;
23     if (n == 2 || n == 3) return true;
24     if (n % 2 == 0) return false;
25     ll d = n - 1;
26     int r = 0;
27     while ((d & 1) == 0) {
28         d >>= 1;
29         ++r;
30     }
31     // 底数集合（对 2^64 以内确定性）
32     static const ll bases[] = {2, 325, 9375, 28178,
450775, 9780504, 1795265022};
33     for (ll a : bases) {
34         if (a % n == 0) continue;
35         ll x = q_pow(a, d, n);
36         if (x == 1 || x == n - 1) continue;
37         bool comp = true;
38         for (int i = 0; i < r - 1; ++i) {
39             x = mul_mod(x, x, n);
40             if (x == n - 1) {
41                 comp = false;
42                 break;
43             }
44         }
45         if (comp) return false;
46     }
47     return true;
48 }
49 // ===== 最大公约数 =====
50 // ===== Pollard-Rho 分解因子（参考你的高效写法）
=====
51 // 小质数表，用于提前试除
52 const vector<ll> small_primes = {2, 3, 5, 7, 11, 13, 17,
19, 23, 29, 31, 37};
53 ll pollard_rho(ll n) {
54     // 先试除小质数
55     for (ll p : small_primes) {
56         if (n % p == 0) return p;
57     }
58
59     static mt19937_64 rng(chrono::steady_clock::now().
time_since_epoch().count());
60     uniform_int_distribution<ll> dist(1, n - 1);
61     auto f = [&](ll x) -> ll {
62         x = mul_mod(x, x, n) + 1;
63         return x >= n ? x - n : x;
64     };
65     ll x = 0, y = 0, tot = 0, p = 1, q, g;
66     for (ll i = 0; (i & 255) || (g = gcd(p, n)) == 1; ++i
) {
67         x = f(x);
68         y = f(f(y));
69         if (x == y) {
70             x = ++tot;
71             y = f(x);
72         }

```

```

73     q = mul_mod(p, (x > y ? x - y : y - x), n);
74     if (q) p = q;
75 }
76 return g;
77 }
78 // ===== 递归分解, 更新最大质因子 =====
79 ll max_factor = 0, n;
80 ll min_factor; // 初始化为 n
81 void factorize_mx(ll n) {
82     if (n <= max_factor || n < 2) return;
83     if (is_prime(n)) {
84         max_factor = max(max_factor, n);
85         return;
86     }
87     ll d = pollard_rho(n);
88     factorize_mx(d);
89     factorize_mx(n / d);
90 }
91 void factorize_mn(ll n) {
92     if (n <= 1) return;
93     if (is_prime(n)) {
94         min_factor = min(min_factor, n);
95         return;
96     }
97     ll d = pollard_rho(n);
98     factorize_mn(d);
99     factorize_mn(n / d);
100 }
101 int main() {
102     cin.tie(nullptr)->ios::sync_with_stdio(false);
103     int T; cin >> T;
104     while (T--) {
105         cin >> n;
106         max_factor = 0, min_factor = n;
107         factorize_mx(n);
108         if (max_factor == n) cout << "Prime\n";
109         else cout << max_factor << '\n';
110     }
111     return 0;
112 }

```

7.10 异或线性基

Listing 60: 异或线性基

```

1 struct LB{
2     vector<ll>a;
3     bool flag;
4     LB(){
5         flag=false;
6         a.resize(63);
7     }
8     bool insert(ll x){
9         for(int i=61;i>=0;i--){
10             if(x>>i&1LL){
11                 if(!a[i]) {
12                     a[i]=x;
13                     return true;
14                 }
15                 x^=a[i];
16             }
17         }

```

```

18         flag=true;
19         return false;
20     }
21     ll ask_max(){
22         ll ans=0;
23         for(int i=61;i>=0;i--){
24             if((ans^a[i])>ans) ans^=a[i];
25         }
26         return ans;
27     }
28     ll ask_min(){
29         if(flag) return 0;
30         for(int i=0;i<=61;i++){
31             if(a[i]) return a[i];
32         }
33         return 0;
34     }
35 };

```

7.11 数论分块

题面简述

给定一个正整数 x , 求 $\sum_{i=1}^x d(i)$, 其中 $d(i)$ 表示 i 的因数个数。

第一步: 转化求和式

由因数个数的定义 $d(i) = \sum_{d|i} 1$, 将求和交换顺序:

$$\sum_{i=1}^x d(i) = \sum_{i=1}^x \sum_{d|i} 1 = \sum_{d=1}^x \left\lfloor \frac{x}{d} \right\rfloor$$

即问题转化为求 $\sum_{d=1}^x \left\lfloor \frac{x}{d} \right\rfloor$ 。

第二步: 整除分块

$\left\lfloor \frac{x}{d} \right\rfloor$ 的取值至多只有 $O(\sqrt{x})$ 种。对于每一种取值 $v = \left\lfloor \frac{x}{l} \right\rfloor$, 所有满足 $\left\lfloor \frac{x}{d} \right\rfloor = v$ 的 d 构成一个连续区间 $[l, r]$, 其中:

$$r = \min\left(\left\lfloor \frac{x}{v} \right\rfloor, x\right)$$

在该区间内 $\left\lfloor \frac{x}{d} \right\rfloor$ 恒为 v , 因此该块的贡献为:

$$v \times (r - l + 1)$$

遍历所有块, 累加贡献即得答案:

$$\text{Ans} = \sum_{\text{block } [l, r]} \left\lfloor \frac{x}{l} \right\rfloor \times (r - l + 1)$$

每次令 $l \leftarrow r + 1$ 跳到下一个块, 直到 $l > x$ 。

复杂度分析

$\left\lfloor \frac{x}{d} \right\rfloor$ 的不同取值个数为 $O(\sqrt{x})$ (当 $d \leq \sqrt{x}$ 时至多 $\sqrt{x}$ 种, 当 $d > \sqrt{x}$ 时 $\left\lfloor \frac{x}{d} \right\rfloor < \sqrt{x}$, 同样至多

$\sqrt{x}$ 种), 因此整除分块的块数为 $O(\sqrt{x})$, 每块内仅需 $O(1)$ 计算。

总时间复杂度为 $O(\sqrt{x})$, 空间复杂度为 $O(1)$ 。

Listing 61: 整除分块

```
1 void solve(){
2     ll n;cin>>n;
3     ll ans=0;
4     for(int l=1,r;l<=n;l=r+1){
5         ll v=(n/l);
6         r=min((n/v),n);
7         ans+=v*(r-l+1);
8     }
9     cout<<ans<<endl;
10 }
```

7.12 莫比乌斯反演

题面简述

给定 N, M , 求满足 $1 \leq x \leq N, 1 \leq y \leq M$ 且 $\gcd(x, y)$ 为质数的有序对 (x, y) 的个数。

多组询问, $T \leq 10^4, N, M \leq 10^7$ 。

第一步: 枚举质数

设答案为 Ans , 直接按定义展开:

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{x=1}^N \sum_{y=1}^M [\gcd(x, y) = p]$$

其中 $\mathcal{P}$ 为质数集合。令 $x = pa, y = pb$, 则:

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{a=1}^{\lfloor N/p \rfloor} \sum_{b=1}^{\lfloor M/p \rfloor} [\gcd(a, b) = 1]$$

第二步: 莫比乌斯反演

利用经典恒等式 $[\gcd(a, b) = 1] = \sum_{d|\gcd(a, b)} \mu(d)$, 代入得:

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{d=1}^{\min(\lfloor N/p \rfloor, \lfloor M/p \rfloor)} \mu(d) \left\lfloor \frac{N}{pd} \right\rfloor \left\lfloor \frac{M}{pd} \right\rfloor$$

第三步: 换元合并

令 $T = pd$, 则 $d = T/p$, 对于固定的 T, p 遍历 T 的所有质因子。交换求和顺序:

$$\text{Ans} = \sum_{T=1}^{\min(N, M)} \left\lfloor \frac{N}{T} \right\rfloor \left\lfloor \frac{M}{T} \right\rfloor \sum_{p|T, p \in \mathcal{P}} \mu\left(\frac{T}{p}\right)$$

定义函数:

$$f(T) = \sum_{p|T, p \in \mathcal{P}} \mu\left(\frac{T}{p}\right)$$

则答案化为:

$$\text{Ans} = \sum_{T=1}^{\min(N, M)} f(T) \left\lfloor \frac{N}{T} \right\rfloor \left\lfloor \frac{M}{T} \right\rfloor$$

第四步: 预处理 f 及其前缀和

$f(T)$ 的预处理方式: 线性筛求出所有质数与 μ 值后, 枚举每个质数 p , 再枚举 p 的倍数 $T = kp$, 将 $\mu(k)$ 累加到 $f(T)$ 上。这一步复杂度为:

$$\sum_{p \in \mathcal{P}} \left\lfloor \frac{n}{p} \right\rfloor \approx O\left(\frac{n \ln \ln n}{1}\right) = O(n \log \log n)$$

随后求出 f 的前缀和 $F(n) = \sum_{T=1}^n f(T)$ 。

第五步: 整除分块回答询问

对每组询问, 利用 $\lfloor N/T \rfloor$ 和 $\lfloor M/T \rfloor$ 的取值分块性质 (整除分块/数论分块), 将 T 的求和区间划分为 $O(\sqrt{N} + \sqrt{M})$ 个块, 每块内 $\lfloor N/T \rfloor$ 和 $\lfloor M/T \rfloor$ 均为常数, 利用前缀和 F 即可 $O(1)$ 求出每块的贡献:

$$\text{Ans} = \sum_{\text{block } [l, r]} \left\lfloor \frac{N}{l} \right\rfloor \left\lfloor \frac{M}{l} \right\rfloor (F(r) - F(l-1))$$

复杂度分析

- 预处理: 线性筛求 μ 和质数表为 $O(n)$; 枚举质数倍数计算 f 为 $O(n \log \log n)$; 前缀和为 $O(n)$ 。总预处理复杂度 $O(n \log \log n)$, 其中 $n = \max(N, M) = 10^7$ 。
- 单次询问: 整除分块为 $O(\sqrt{n})$ 。
- 总复杂度: $O(n \log \log n + T\sqrt{n})$ 。

对于本题 $n = 10^7, T = 10^4$:

- 预处理: $10^7 \times \log \log (10^7) \approx 10^7 \times 3 \approx 3 \times 10^7$ 。
- 查询: $10^4 \times \sqrt{10^7} \approx 10^4 \times 3162 \approx 3.16 \times 10^7$ 。

总操作量约 6×10^7 , 可以通过。空间复杂度为 $O(n)$ 。

Listing 62: 莫比乌斯反演

```
1 //P2398 GCD-SUM
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 const int N=1e5+5;
7 ll p[N],vis[N],cnt;
```

```

8 ll mu[N],pre[N],phi[N];
9 void init(int n){
10     mu[1]=phi[1]=1;
11     for(int i=2;i<=n;i++){
12         if(!vis[i]){
13             p[cnt++]=i;
14             mu[i]=-1;
15             phi[i]=i-1;
16         }
17         for(int j=0;p[j]<=n;j++){
18             int m=i*p[j];
19             vis[m]=1;
20             if(i%p[j]==0){
21                 mu[m]=0;
22                 phi[m]=phi[i]*p[j];
23                 break;
24             }
25             phi[m]=phi[i]*(p[j]-1);
26             mu[m]=-mu[i];
27         }
28     }
29     for(int i=1;i<=n;i++) pre[i]=pre[i-1]+phi[i];
30 }
31 int main(){
32     cin.tie(0)->ios::sync_with_stdio(false);
33     ll ans=0;
34     ll n;cin>>n;
35     init(n+5);
36     for(int l=1,r;l<=n;l=r+1){
37         ll v=n/l;
38         r=n/v;
39         ans+=v*(pre[r]-pre[l-1]);
40     }
41     cout<<ans<<endl;
42     return 0;
43 }

```

7.13 exgcd

Listing 63: exgcd

```

1 //P1516青蛙的约会
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 ll exgcd(ll a,ll b,ll &x,ll &y){
7     if(!b){
8         x=1,y=0;
9         return a;
10    }
11    ll d=exgcd(b,a%b,y,x);
12    y-=a/b*x;
13    return d;
14 }
15 void solve(){
16     int x,y,m,n,l;
17     cin>>x>>y>>m>>n>>l;
18     ll X,Y;
19     ll d=exgcd(n-m,l,X,Y);
20     if((x-y)%d cout<<"Impossible"<<endl;
21     else{

```

```

22         ll s=(x-y)/d;
23         ll mod=abs(l/d);
24         X*=s;
25         Y*=s;
26         cout<<(X%mod+mod)%mod<<endl;
27     }
28 }

```

8 计算几何

8.1 极角排序

Listing 64: 极角排序求夹角最近两向量

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 typedef long double ld;
6 const ll INF=1e18;
7 const ld pi=acos(-1);
8 struct Point{
9     ll x,y;
10    int id,idx;
11    Point(ll x,ll y,int idx):x(x),y(y),idx(idx){
12        if(x>=0&&y>=0) id=1;
13        else if(x>=0&&y<0) id=2;
14        else if(x<0&&y<0) id=3;
15        else id=4;
16    };
17    Point(){};
18    friend bool operator<(Point A,Point B){
19        if(A.id!=B.id) return A.id<B.id;
20        return A.x*B.y-A.y*B.x<0;
21    }//极角排序
22 };
23 void solve(){
24     int n;cin>>n;
25     vector<Point>a(n+2);
26     for(int i=1;i<=n;i++){
27         ll x,y;
28         cin>>x>>y;
29         a[i]=Point(x,y,i);
30     }
31     sort(a.begin()+1,a.begin()+1+n);
32     ll x=0,y=0;
33     auto calc=[&](Point &A,Point &B) ->ld {
34         ld cross=A.x*B.y-A.y*B.x;
35         ld dot=A.x*B.x+A.y*B.y;
36         return fabs1(atan2l(cross,dot));
37     };//较小精度损失的计算角度
38     ++n;
39     a[n]=a[1];//让 n和1比一次
40     ld mn=INF;
41     for(int i=1;i<n;i++){
42         ld res=calc(a[i],a[i+1]);
43         if(res<mn){
44             x=a[i].idx,y=a[i+1].idx;
45             mn=res;
46         }
47     }
48     cout<<x<<' '<<y<<endl;

```

```

49 }
50 int main(){
51     cin.tie(0)->ios::sync_with_stdio(false);
52     solve();
53     return 0;
54 }

```

9 Other Trick

9.1 多路归并 (含二分分解)

题面简述

给定两个长度为 N 的单调不降序列 A, B , 从 A, B 中各取一个数相加可得到 N^2 个和, 求这 N^2 个和中最小的 N 个。

第一步: 小根堆初始化与去重策略

由于序列 A, B 均单调不降, 最小的和必然是 $A_1 + B_1$ (下标从 1 开始)。我们使用一个小根堆 (优先队列) 维护候选和, 堆中元素为三元组 (sum, i, j) , 表示 $A_i + B_j$ 的值及其对应下标。

为避免重复访问同一 (i, j) 对, 采用如下策略:

- 初始将 $(A_1 + B_1, 1, 1)$ 入堆, 并标记 $(1, 1)$ 为已访问。
- 每次取出堆顶 (sum, i, j) 加入答案后:
 1. 若 $j + 1 \leq N$ 且 $(i, j + 1)$ 未访问, 将 $(A_i + B_{j+1}, i, j + 1)$ 入堆并标记。
 2. 若 $j = 1$ (当前是第 i 行首个元素) 且 $i + 1 \leq N$, 将 $(A_{i+1} + B_j, i + 1, j)$ 入堆并标记。

该策略确保每个 (i, j) 仅入堆一次, 避免重复计算。

第二步: 提取前 N 个最小和

重复执行 N 次以下操作:

1. 取出堆顶元素, 将其和加入答案数组。
2. 按上述策略将后续候选和入堆。

最终答案数组即为所求的 N 个最小和。

正确性说明

由于 A, B 单调不降, 对任意 (i, j) 有 $A_i + B_j \leq A_{i+1} + B_j$ 且 $A_i + B_j \leq A_i + B_{j+1}$ 。小根堆保证每次取出当前最小候选和, 去重策略确保无重复入堆, 且不遗漏任何可能的最小和。

Listing 65: 堆

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 typedef pair<ll, pair<int, int>> P;

```

```

6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int N; cin >> N;
11    vector<ll> A(N), B(N);
12    for (int i = 0; i < N; ++i) cin >> A[i];
13    for (int i = 0; i < N; ++i) cin >> B[i];
14    priority_queue<P, vector<P>, greater<P>> pq;
15    pq.push({A[0] + B[0], {0, 0}});
16    for (int k = 0; k < N; ++k) {
17        P top = pq.top();
18        pq.pop();
19        ll sum = top.first;
20        int i = top.second.first;
21        int j = top.second.second;
22        if (k > 0) cout << ' ';
23        cout << sum;
24        if (j + 1 < N) {
25            pq.push({A[i] + B[j+1], {i, j+1}});
26        }
27        if (j == 0 && i + 1 < N) {
28            pq.push({A[i+1] + B[j], {i+1, j}});
29        }
30    }
31    cout << endl;
32    return 0;
33 }

```

第一步：二分查找确定第 N 小的和的值 x

定义函数 $\text{calc}(mid)$ ，用于计算序列 A, B 中满足 $A_i + B_j \leq mid$ 的数对 (i, j) 的数量。利用单调性，该函数可通过双指针法在 $O(N)$ 时间内实现：

- 初始化指针 $j = N - 1$ （指向 B 的末尾）。
- 遍历 A 的每个元素 A_i ：
 - 若 $A_i + B_j > mid$ ，则 j 左移（减小 j ）。
 - 此时 B_0 到 B_j 与 A_i 的和均 $\leq mid$ ，贡献 $j + 1$ 个计数。

二分查找的初始区间设为 $[l, r]$ ，其中 $l = A_0 + B_0$ （最小和）， $r = A_{N-1} + B_{N-1}$ （最大和）。二分过程如下：

1. 取中点 $mid = l + \frac{r-l}{2}$ 。
2. 若 $\text{calc}(mid) \geq N$ ，说明第 N 小的和 $\leq mid$ ，令 $r = mid$ 。
3. 否则，说明第 N 小的和 $> mid$ ，令 $l = mid + 1$ 。
4. 当 $l = r$ 时， $x = l$ 即为第 N 小的和的值。

第二步：收集所有严格小于 x 的和

再次利用双指针法，遍历序列 A ：

- 对每个 A_i ，找到最大的 j 使得 $A_i + B_j < x$ 。
- 将 $A_i + B_0, A_i + B_1, \dots, A_i + B_j$ 全部加入结果数组 res 。

第三步：补充等于 x 的和至 N 个

计算还需补充的数量 $\text{need} = N - \text{res.size}()$ 。若 $\text{need} > 0$ ，再次遍历序列 A ：

- 对每个 A_i ，找到最大的 j 使得 $A_i + B_j \leq x$ 。
- 从 j 开始向左遍历，若 $A_i + B_k == x$ ，则将 x 加入 res ，并减少 need ，直到 $\text{need} = 0$ 。

第四步：排序并输出

对结果数组 res 进行排序，取前 N 个元素依次输出，即为所求的 N 个最小和。

正确性说明

二分查找确保找到的 x 是满足“和 $\leq x$ 的数对数量 $\geq N$ ”的最小值，因此 x 必为第 N 小的和。通过先收集所有小于 x 的和，再补充等于 x 的和，确保结果数组包含恰好 N 个最小和，排序后顺序正确。

复杂度分析

- 时间复杂度：
 - 二分查找部分：共进行 $O(\log S)$ 次二分（ S 为和的范围，约 2×10^9 ），每次 calc 函数耗时 $O(N)$ ，总耗时 $O(N \log S)$ 。
 - 收集与补充和的部分：两次双指针遍历均为 $O(N)$ ，最后排序为 $O(N \log N)$ 。
 - 综上，总时间复杂度为：

$$O(N \log N + N \log S)$$

- 空间复杂度：结果数组 res 需 $O(N)$ 空间，总空间复杂度为 $O(N)$ 。

Listing 66: 二分

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 long long calc(const vector<long long>& a, const vector<
4     long long>& b, long long mid) {
5     int n = a.size();

```

```

5   long long cnt = 0;
6   int j = n - 1;
7   for (int i = 0; i < n; ++i) {
8       while (j >= 0 && a[i] + b[j] > mid) j--;
9       cnt += j + 1;
10  }
11  return cnt;
12 }
13 int main() {
14     cin.tie(nullptr)->ios::sync_with_stdio(false);
15     int n;cin >> n;
16     vector<long long> a(n), b(n);
17     for (int i = 0; i < n; ++i) cin >> a[i];
18     for (int i = 0; i < n; ++i) cin >> b[i];
19     long long l = a[0] + b[0], r = a.back() + b.back();
20     while (l < r) {
21         long long mid = l + (r - l) / 2;
22         if (calc(a, b, mid) >= n) {
23             r = mid;
24         } else {
25             l = mid + 1;
26         }
27     }
28     long long x = l;
29     vector<long long> res;
30     res.reserve(n);
31     int j = n - 1;
32     for (int i = 0; i < n; ++i) {
33         while (j >= 0 && a[i] + b[j] >= x) j--;
34         for (int k = 0; k <= j; ++k) {
35             res.push_back(a[i] + b[k]);
36         }
37     }
38     int need = n - res.size();
39     j = n - 1;
40     for (int i = 0; i < n && need > 0; ++i) {
41         while (j >= 0 && a[i] + b[j] > x) j--;
42         int k = j;
43         while (k >= 0 && a[i] + b[k] == x && need > 0) {
44             res.push_back(x);
45             need--;
46             k--;
47         }
48     }
49     sort(res.begin(), res.end());
50     for (int i = 0; i < n; ++i) {
51         cout << res[i] << " ";
52     }
53     return 0;
54 }

```

9.2 使用栈做递归防溢出

Listing 67: 原始代码

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  typedef long long ll;
5  const ll INF=1e18;
6  const int N=1e7;
7  void solve(){

```

```

8      int n;cin>>n;
9      vector<ll>a(n+1);
10     vector<vector<int>>>G(n+1);
11     vector<ll>f(n+1);
12     ll sum=0;
13     for(int i=1;i<=n;i++) cin>>a[i],sum+=a[i];
14     for(int i=1;i<=n;i++){
15         int x,y;cin>>x>>y;
16         G[x].push_back(y);
17         G[y].push_back(x);
18     }
19     vector<ll>dp(n+1);
20     vector<ll>siz(n+1);
21     auto dfs=[&](auto &&dfs,int u,int fa) ->void {
22         dp[u]=INF;
23         f[u]=a[u];
24         siz[u]=1;
25         for(int v:G[u]){
26             if(v==fa) continue;
27             dfs(dfs,v,u);
28             f[u]+=f[v];
29             siz[u]+=siz[v];
30             dp[u]=min(dp[u],dp[v]);
31         }
32         dp[u]=min(dp[u],f[u]/siz[u]);
33     };
34     dfs(dfs,1,0);
35     for(int i=1;i<=n;i++) cout<<dp[i]<<' ';
36     cout<<endl;
37 }
38 int main(){
39     cin.tie(0)->ios::sync_with_stdio(false);
40     int size(256<<20);
41     __asm__ ( "movq %0, %%rsp\n"::"r"((char*)malloc(size)
42     +size));
43     int T=1;cin>>T;
44     while(T-- solve();
45     exit(0);
46     return 0;
47 }

```

Listing 68: 栈递归版

```

1  struct Statu{
2      int u,fa,op;//当前节点,父节点,操作方式
3  };
4  int x,y;
5  void solve() {
6      int n; cin>>n;
7      vector<ll>a(n+1),f(n+1),dp(n+1);
8      vector<Statu>st(n+1);
9      vector<int>siz(n+1);
10     vector<vector<int>>>G(n+1);
11     for(int i=1;i<=n;i++) cin>>a[i];
12     for(int i=1;i<=n;i++){
13         cin>>x>>y;
14         G[x].push_back(y);
15         G[y].push_back(x);
16     }
17     st.push_back({1,0,0});
18     while(!st.empty()){
19         Statu &cur=st.back();
20         int u=cur.u,fa=cur.fa;

```

```

21     if(!cur.op){
22         dp[u]=INF;
23         f[u]+=a[u];
24         siz[u]++;
25         cur.op=1;
26         for(int v:G[u]){
27             if(v==fa) continue;
28             st.push_back({v,u,0});
29         }
30     }
31     else{
32         for(int v:G[u]){
33             if(v==fa) continue;
34             dp[u]=min(dp[u],dp[v]);
35             siz[u]+=siz[v];
36             f[u]+=f[v];
37         }
38         dp[u]=min(dp[u],f[u]/siz[u]);
39         st.pop_back();
40     }
41 }
42 for(int i=1;i<=n;i++) cout<<dp[i]<<' ';
43 cout<<endl;
44 }

```

9.3 Log Trick

前缀 GCD ,OR , AND 等操作变化的级别是呈现对数级别的。

问题简述 (Hdu 5869)

给定一个包含 N 个正整数 $a_1, a_2, \dots, a_{N-1}, a_N$ 的数组 a ；定义 a 的子数组为 a_1 到 a_N 之间的连续区间。换句话说，对于 $1 \leq i \leq j \leq N$ 而言， $a_i, a_{i+1}, \dots, a_{j-1}, a_j$ 就是 a 的一个子数组。对于形如 (L, R) 的查询，请回答区间 $[L, R]$ 内所有子数组产生的不同 GCD 值的数量。

数据保证：

- $1 \leq N, Q \leq 100000$
- $1 \leq a_i \leq 1000000$

核心性质观察

对于数组中的每个位置 i （作为子数组的右端点），所有以 i 为右端点的子数组的 GCD 具有以下性质：

- 这些 GCD 的数量是 $O(\log a_i)$ 级别的。因为每次将新的 a_i 与前序 GCD 取新的 GCD 时，数值至少减半（或保持不变），因此不同 GCD 的数量随右端点递增呈对数级增长。

- 可以通过维护一个列表来记录这些 GCD 及其对应的最左左端点。例如，列表中的元素为 (g, l) ，表示以 i 为右端点、GCD 为 g 的子数组的左端点起始位置为 l 。

离线处理与查询排序

1. 将所有查询按右端点 R 从小到大排序，并记录每个查询的原始索引，以便最后按输入顺序输出答案。
2. 从左到右遍历数组，每次处理到位置 i 时，同步处理所有 $R = i$ 的查询。

维护以 i 为右端点的 GCD 列表

当遍历到位置 i 时，按以下步骤维护当前的 GCD 列表：

1. 初始化临时列表 cur ，先加入 (a_i, i) （对应子数组 $[i, i]$ ，其 GCD 为 a_i ）。
2. 取上一次处理 $i - 1$ 时得到的列表 $prev$ ，对其中每个元素 (g, l) ，计算新的 GCD 值 $g' = \gcd(g, a_i)$ ，并将 (g', l) 加入 cur 。
3. 合并 cur 中相同的 GCD 值：对相同的 g' ，仅保留最小的左端点 l （因为更小的 l 对应覆盖范围更广的子数组）。合并后的列表作为新的 $prev$ ，用于处理 $i + 1$ 。

利用树状数组维护不同 GCD 的计数

为了快速查询区间 $[L, R]$ 内的不同 GCD 数量，我们使用树状数组维护标记信息：

- 定义数组 $las[g]$ ，表示 GCD 值 g 最后一次被标记的左端点位置，初始为 0。
- 对于当前 cur 列表中的每个元素 (g, l) ：
 1. 若 $las[g] \neq 0$ ，说明之前在位置 $las[g]$ 标记过 g ，需在树状数组中对 $las[g]$ 位置执行单点减 1 操作（取消旧标记）。
 2. 在树状数组中对位置 l 执行单点加 1 操作（标记新的最左位置）。
 3. 更新 $las[g] = l$ 。
- 此时，树状数组中区间 $[L, R]$ 的和即为查询 $[L, R]$ 的答案：每个 GCD 值仅被标记一次，且仅当标记位置 $l \in [L, R]$ 时被计数。

复杂度分析

• 时间复杂度:

- 维护 GCD 列表: 每个元素 a_i 对应的列表长度为 $O(\log a_i)$, 总耗时 $O(N \log A)$ (A 为 a_i 的最大值, 约 10^6)。
- 树状数组操作: 每次单点更新和区间求和均为 $O(\log N)$, 总耗时 $O((N+Q) \log N)$ 。
- 综上, 总时间复杂度为:

$$O(N \log A + Q \log N)$$

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 struct BIT {
6     vector<ll>a;
7     int n;
8     BIT(int n):n(n+1){ a.resize(n<<1);}
9     void add(int x,ll k) {
10         while (x<=n) {
11             a[x]+=k;
12             x+=x&-x;
13         }
14     }
15     int qry(int x) {
16         ll ans=0;
17         while (x) {
18             ans+=a[x];
19             x&=x-1;
20         }
21         return ans;
22     }
23 };
24 struct Query{
25     int L, R, idx;
26     bool operator<(const Query& other) const {
27         return R < other.R;
28     }
29 };
30 int n,q;
31 void solve(){
32     vector<int>a(n+1);
33     for (int i=1;i<=n;i++) cin>>a[i];
34     vector<Query>qs(q);
35     for (int i=0;i<q;i++) {
36         cin>>qs[i].L>>qs[i].R;
37         qs[i].idx=i;
38     }
39     sort(qs.begin(),qs.end());
40     BIT S(n+1);
41     vector<pair<int, int>> prev;
42     map<int,int>las;
43     vector<int>ans(q);
44     int q_ptr=0;
45     for (int i=1;i<=n;i++) {
46         vector<pair<int,int>>cur;
```

```
47         cur.emplace_back(a[i],i);
48         for (auto& p:prev) cur.emplace_back(gcd(p.first,a
49 [i]),p.second);
50         vector<pair<int,int>> merged;
51         for (auto& p:cur) {
52             if (merged.empty()||merged.back().first!=p.
53 first) {
54                 merged.push_back(p);
55             }
56         }
57         for(auto& [g,l] :merged) {
58             if (las.count(g)) S.add(las[g],-1);
59             S.add(l,1);
60             las[g]=l;
61         }
62         while(q_ptr<q&&qs[q_ptr].R==i) {
63             int L=qs[q_ptr].L;
64             int R=qs[q_ptr].R;
65             int idx=qs[q_ptr].idx;
66             ans[idx]=S.qry(R)-S.qry(L - 1);
67             q_ptr++;
68         }
69         prev=move(merged);
70     }
71     for (int x:ans) cout<<x<<endl;
72 }
73 int main() {
74     cin.tie(nullptr)->ios::sync_with_stdio(false);
75     while(cin>>n>>q) solve();
76     return 0;
77 }
```

9.4 异或前缀模 4 小规律

设 $S(n) = 1 \oplus 2 \oplus 3 \oplus \dots \oplus n$, 则有:

$$S(n) = \begin{cases} n, & n \bmod 4 = 0 \\ 1, & n \bmod 4 = 1 \\ n+1, & n \bmod 4 = 2 \\ 0, & n \bmod 4 = 3 \end{cases}$$

9.5 曼哈顿距离与切比雪夫距离的相互转化

曼哈顿距离转切比雪夫距离

设原坐标系下两点坐标为 (x_1, y_1) 和 (x_2, y_2) 。
定义变换:

$$u = x + y, \quad v = x - y$$

则在新坐标系 (u, v) 下, 曼哈顿距离等于切比雪夫距离:

$$|x_1 - x_2| + |y_1 - y_2| = \max(|u_1 - u_2|, |v_1 - v_2|)$$

因此, 处理最大曼哈顿距离问题时, 只需在新坐标系下维护 u 和 v 的 min / max 值, 即可 $O(1)$ 得到结果。

切比雪夫距离转曼哈顿距离

设切比雪夫坐标系下两点坐标为 (u_1, v_1) 和 (u_2, v_2) 。定义逆变换：

$$x = \frac{u+v}{2}, \quad y = \frac{u-v}{2}$$

则原坐标系下的曼哈顿距离等于切比雪夫距离：

$$\max(|u_1 - u_2|, |v_1 - v_2|) = |x_1 - x_2| + |y_1 - y_2|$$

注意：若原坐标为整数，则 u 与 v 同奇偶，上式中的除法结果仍为整数或半整数，但距离公式依然成立。实际编程时可先全部乘以 2 避免浮点数。

9.6 判定可重集合是否相等

定义

可重集合 (Multiset)：是指允许同一元素出现多次的集合。

问题描述

给定数组 a ，多次询问：判断区间 $[L_1, R_1]$ 与 $[L_2, R_2]$ 所对应的可重集合是否相等。设每次询问的两个可重集合分别为 S 和 T 。

朴素做法

直接的做法包括：

- 将两个区间内的元素排序逐一比较。
- 使用哈希表统计元素出现次数。

设 $|S| \leq |T|$ ，单次询问时间复杂度为 $\mathcal{O}(|S| \cdot \log |S|)$ 。

随机化做法（随机赋权哈希）

定理

如果两个可重集合相等，那么两个集合的元素和相等。但仅靠元素和无法判定，反例： $S = \{2, 4\}$ ， $T = \{3, 3\}$ 。

改进思路

为每一种元素 x 随机赋予一个权值 $w(x)$ ，定义集合的哈希值为：

$$H(S) = \sum_{x \in S} w(x)$$

判断两个集合是否相等，只需比较 $H(S)$ 和 $H(T)$ 。

正确性分析

- 若 $S = T$ ，则 $H(S) = H(T)$ 。
- 若 $S \neq T$ ，设 $\text{cnt}_S(x), \text{cnt}_T(x)$ 分别表示元素 x 在集合中的出现次数，则

$$\sum (\text{cnt}_S(x) - \text{cnt}_T(x)) \cdot w(x) = 0$$

这是一个关于随机变量 $w(x)$ 的线性组合。在 $w(x)$ 独立随机且值域足够大的情况下，上式成立的概率极低。

其他哈希形式

也可以定义 $H(S) = \prod_{x \in S} w(x)$ ，同样可用于判定可重集合相等（本质仍为随机映射）。

区间查询实现

- 不带修：使用前缀和维护，单次查询 $\mathcal{O}(1)$ 。
- 待修：使用树状数组/线段树维护区间和，单次查询 $\mathcal{O}(\log n)$ 。

扩展问题

给定数组 a ，多次询问：判断区间 $[L_1, R_1]$ 中所有元素加上 k 后，是否与区间 $[L_2, R_2]$ 的可重集合相等。

设常数 B ，定义哈希函数：

$$H(S) = \sum B^{a_i}$$

则有：

$$H([L, R]) = \sum_{i=L}^R B^{a_i}$$

区间整体加上 k 后：

$$H'([L, R]) = \sum_{i=L}^R B^{a_i+k} = B^k \cdot \sum_{i=L}^R B^{a_i} = B^k \cdot H([L, R])$$

因此只需判断 $B^k \cdot H([L_1, R_1])$ 与 $H([L_2, R_2])$ 的关系即可。